CSE4007
Artificial Intelligence

Lecture 10: Generative Models Part 2

Variational Autoencoder (VAE)

Department of Data Science
Hanyang University

Variational Autoencoders

Variational Autoencoders

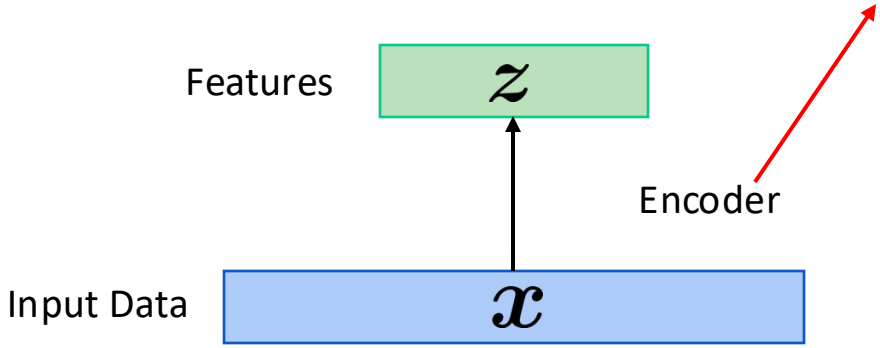
(Regular, non-variational) Autoencoders

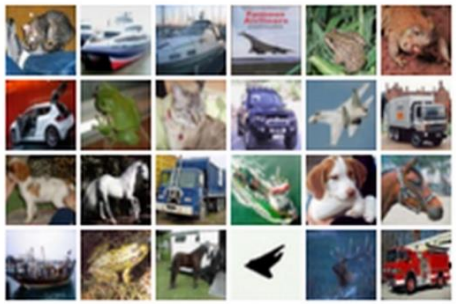
- Unsupervised method for learning feature vectors from raw data $\boldsymbol{x}$, without any labels
  - Features should extract **useful information** (maybe object identities, properties, scene type, etc.) that we can use for downstream tasks

Originally: Linear + nonlinearity (sigmoid)

Later: Deep, fully-connected

Later: ReLU CNN





Input Data

(Regular, non-variational) Autoencoders

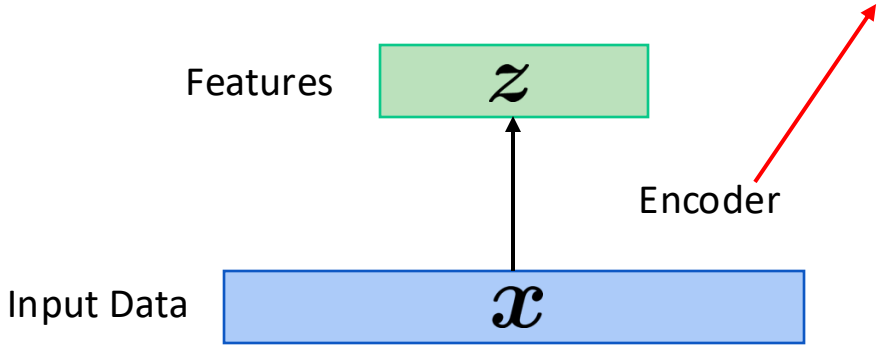
- Problem: How can we learn this feature transform from raw data?

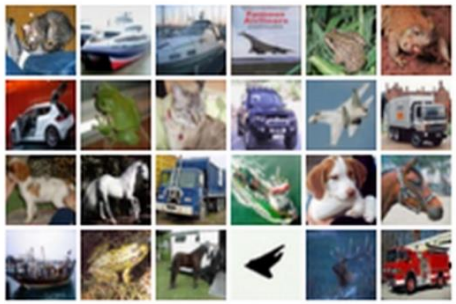
- Features should extract **useful information** (maybe object identities, properties, scene type, etc.) that we can use for downstream tasks
- But, we can't observe features!

Originally: Linear + nonlinearity (sigmoid)

Later: Deep, fully-connected

Later: ReLU CNN





Input Data

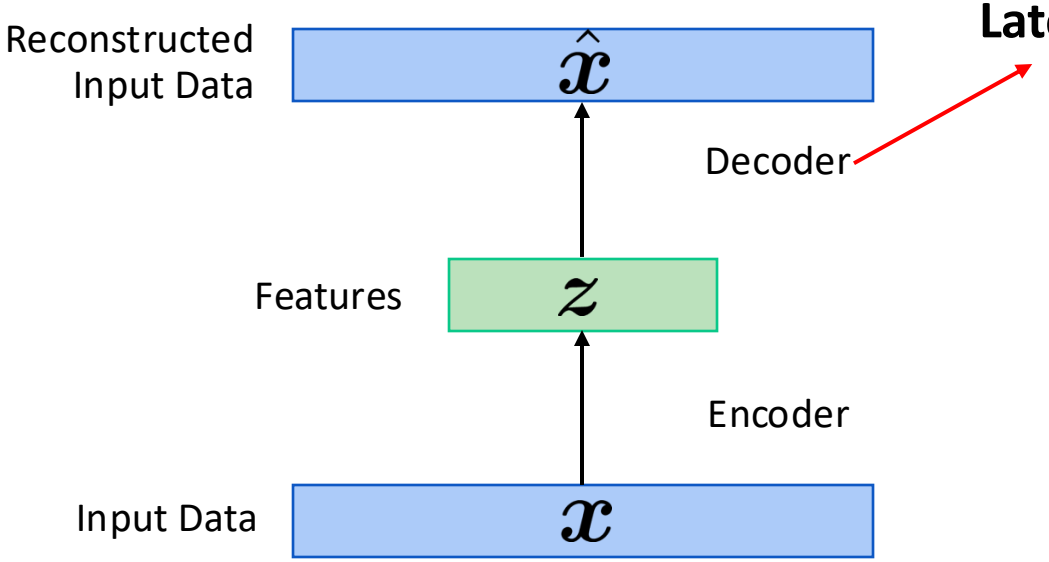
(Regular, non-variational) Autoencoders

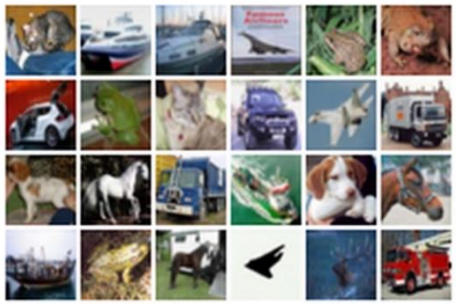
- Problem: How can we learn this feature transform from raw data?
- **Idea**: Use the features to <u>reconstruct</u> the input with a **decoder**
- "Autoencoding" = encoding itself

Originally: Linear + nonlinearity (sigmoid)

Later: Deep, fully-connected

Later: ReLU CNN

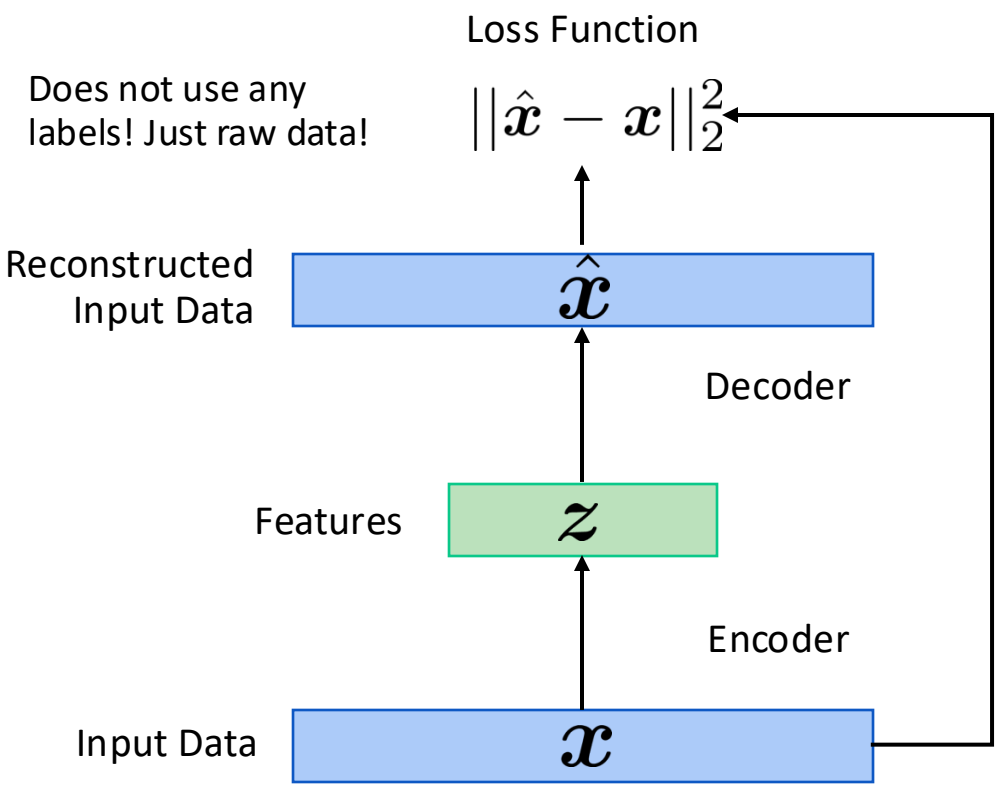


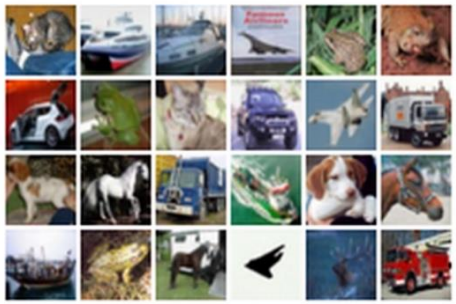


Input Data

(Regular, non-variational) Autoencoders

- **Loss**: L2 distance between input and reconstructed data

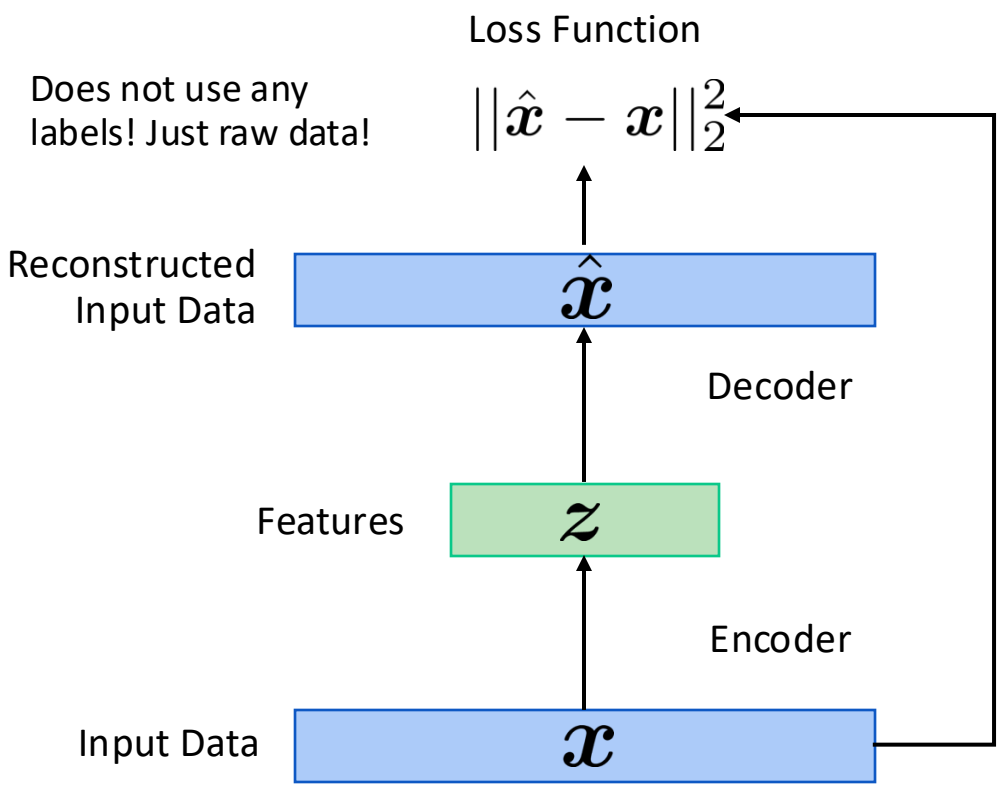


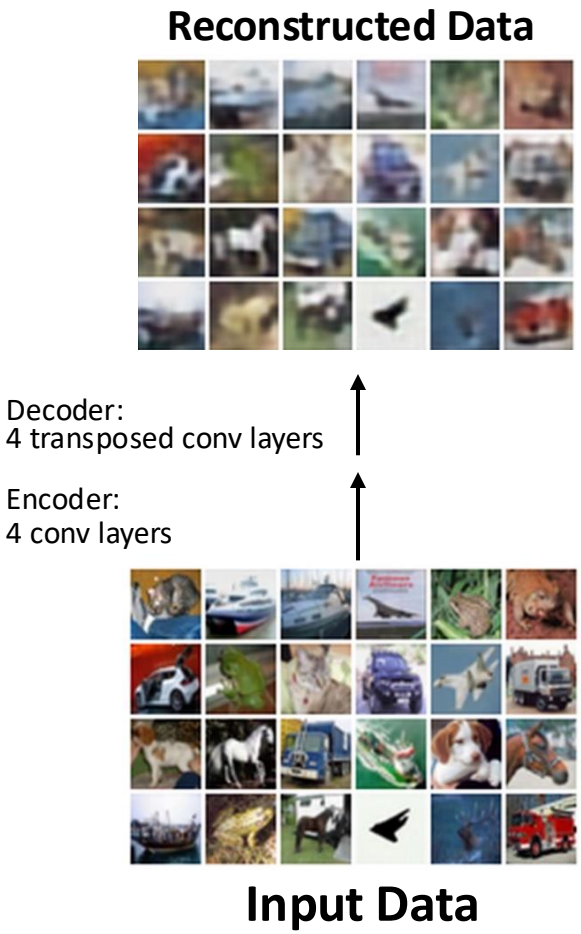


Input Data

(Regular, non-variational) Autoencoders

- **Loss**: L2 distance between input and reconstructed data





(Regular, non-variational) Autoencoders

- **Loss**: L2 distance between input and reconstructed data

Loss Function

Does not use any labels! Just raw data!

$$||\hat{\boldsymbol{x}} - \boldsymbol{x}||_2^2$$

Reconstructed Input Data

$\hat{\boldsymbol{x}}$

Decoder

Features need to be **lower dimensional** than the data

Features

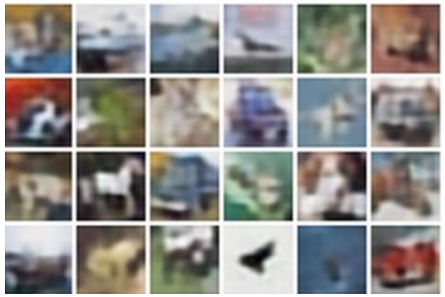
$\boldsymbol{z}$

Encoder

Input Data

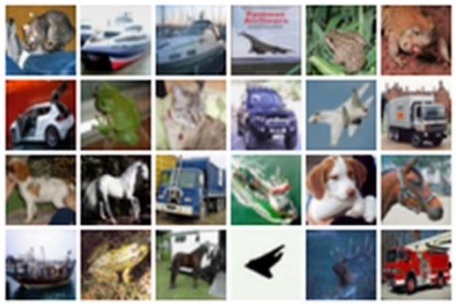
$\boldsymbol{x}$

Reconstructed Data



Decoder:
4 transposed conv layers

Encoder:
4 conv layers



Input Data

(Regular, non-variational) Autoencoders

- **Loss**: L2 distance between input and reconstructed data

Loss Function

Does not use any labels! Just raw data!

$$||\hat{\boldsymbol{x}} - \boldsymbol{x}||_2^2$$

Reconstructed Input Data $\hat{\boldsymbol{x}}$

Decoder

Features need to be **lower dimensional** than the data

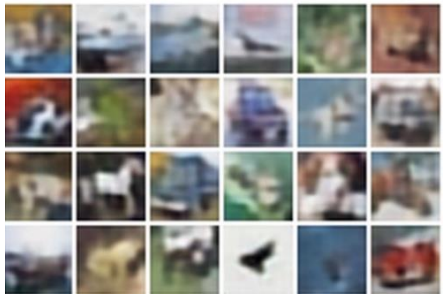
Why?

Features $\boldsymbol{z}$

Encoder

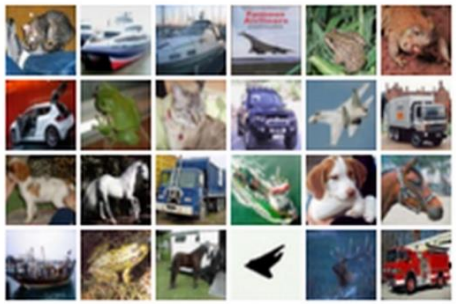
Input Data $\boldsymbol{x}$

Reconstructed Data



Decoder:
4 transposed conv layers

Encoder:
4 conv layers



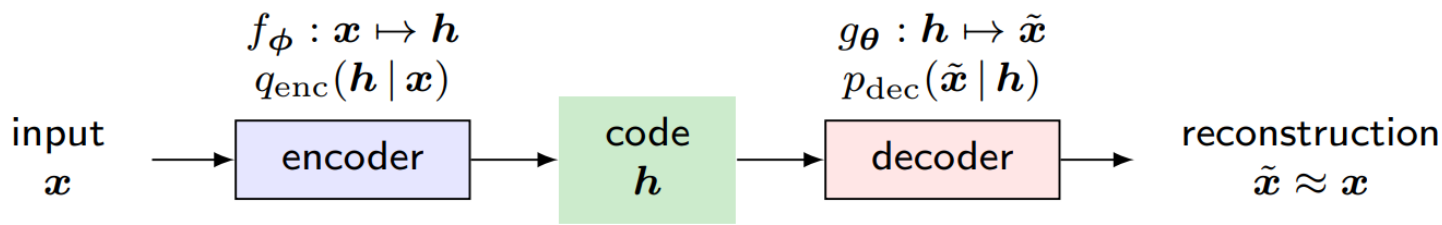
Input Data

Autoencoder (AE)

- Neural network trained to reconstruct input $\boldsymbol{x}$ as output $\tilde{\boldsymbol{x}}$

  i.e. $\boldsymbol{x} \approx \tilde{\boldsymbol{x}}$
  - Learns code (=efficient representation of input data)
  - Unsupervised (no label)
- architecture

	deterministic	stochastic	parameterized by
encoder	$f_{\boldsymbol{\phi}}(\boldsymbol{x}) = \boldsymbol{h}$	$q_{\text{enc}}(\boldsymbol{h} \mid \boldsymbol{x})$	$\boldsymbol{\phi}$
decoder	$g_{\boldsymbol{\theta}}(\boldsymbol{h}) = \tilde{\boldsymbol{x}}$	$p_{\text{dec}}(\tilde{\boldsymbol{x}} \mid \boldsymbol{h})$	$\boldsymbol{\theta}$



  - dim($\boldsymbol{x}$) > dim(**h**): undercomplete ⇒ capture important features of input
  - dim($\boldsymbol{x}$) < dim(**h**): overcomplete

Need for regularization

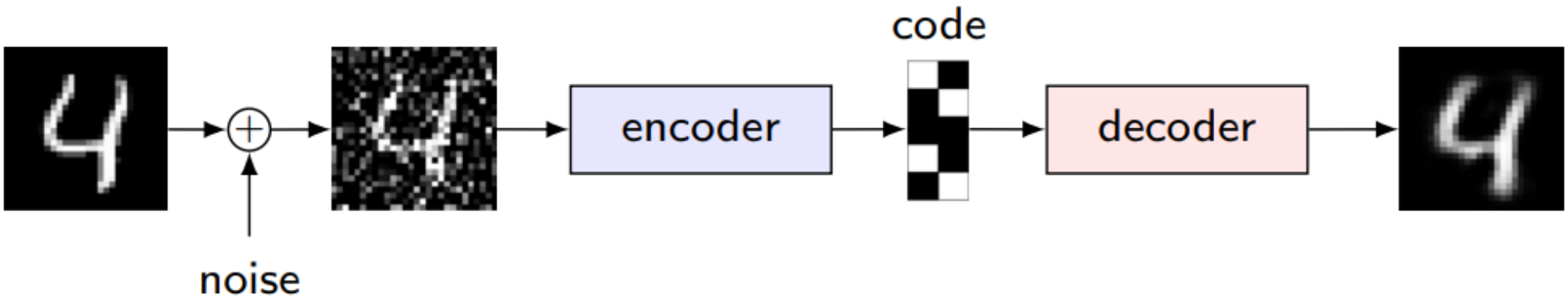
- Encoder/decoder with too much capacity
  - Simply learn to copy input to output
  - Learn nothing useful about data distribution

- Regularizing AE
  - Limit the model capacity
  - Use a loss encouraging desirable properties

- Desirable properties
  - Robustness to noise/missing inputs (e.g. denoising autoencoder)
  - Sparse representation (e.g. sparse autoencoder)
  - Similar input → similar representation (e.g. contractive autocendoer)

Denoising Autoencoder (DAE)

- One way to force AE to learn useful features
  - Add noise to inputs
  - Then train AE to recover original (noise-free) input

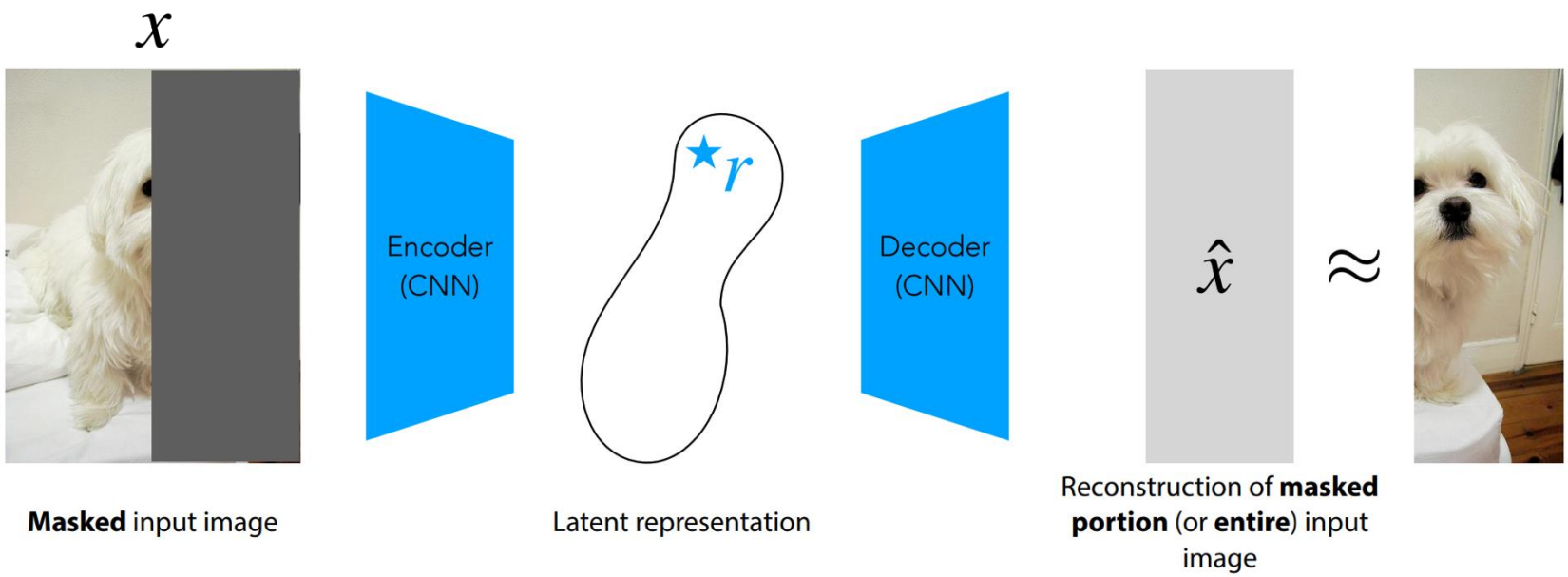


- Types of noise
  - Gaussian noise
  - Dropout: randomly switched off inputs

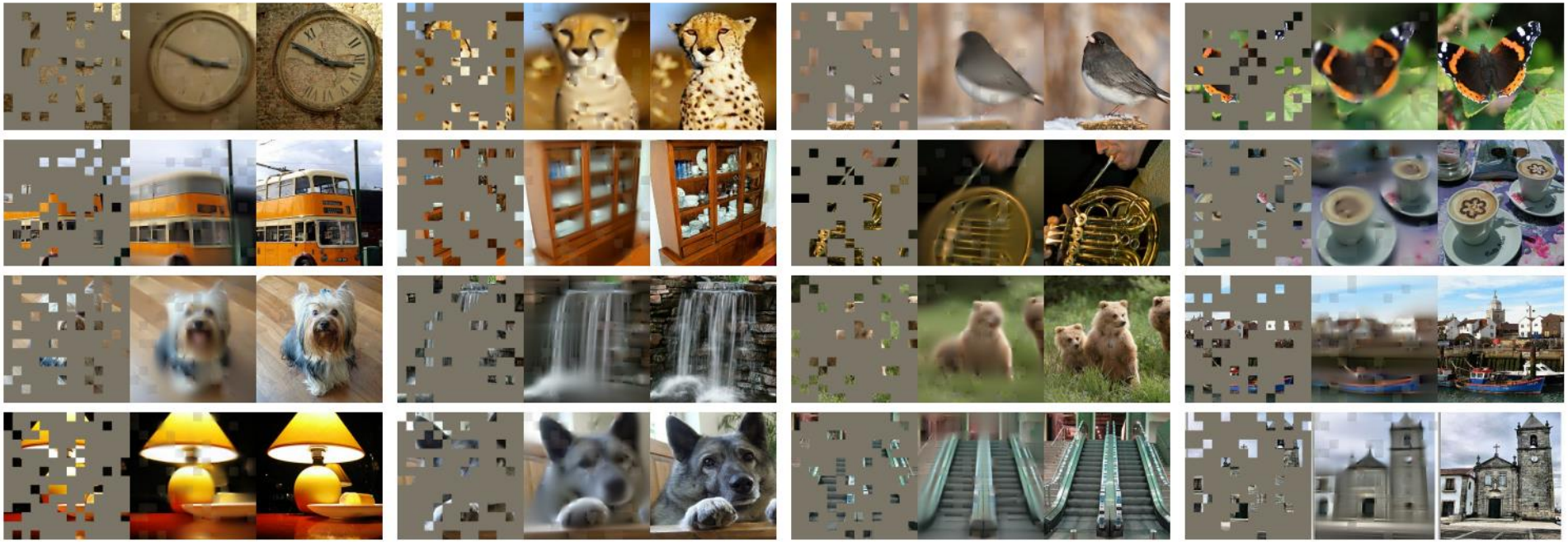
- BERT: a kind of denoising autoencoder for NLP

MAE (Masked Autoencoders)

- Another way to force AE to learn useful features
  - Make a more difficult learning task to encourage the encoder to extract more meaningful features

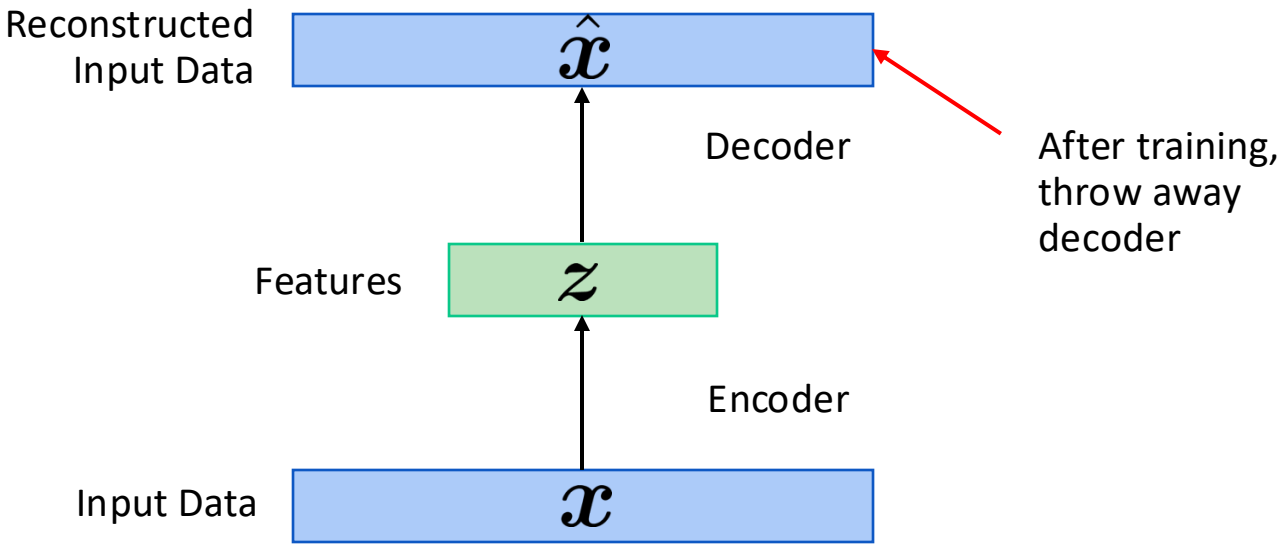


MAE (Masked Autoencoders)



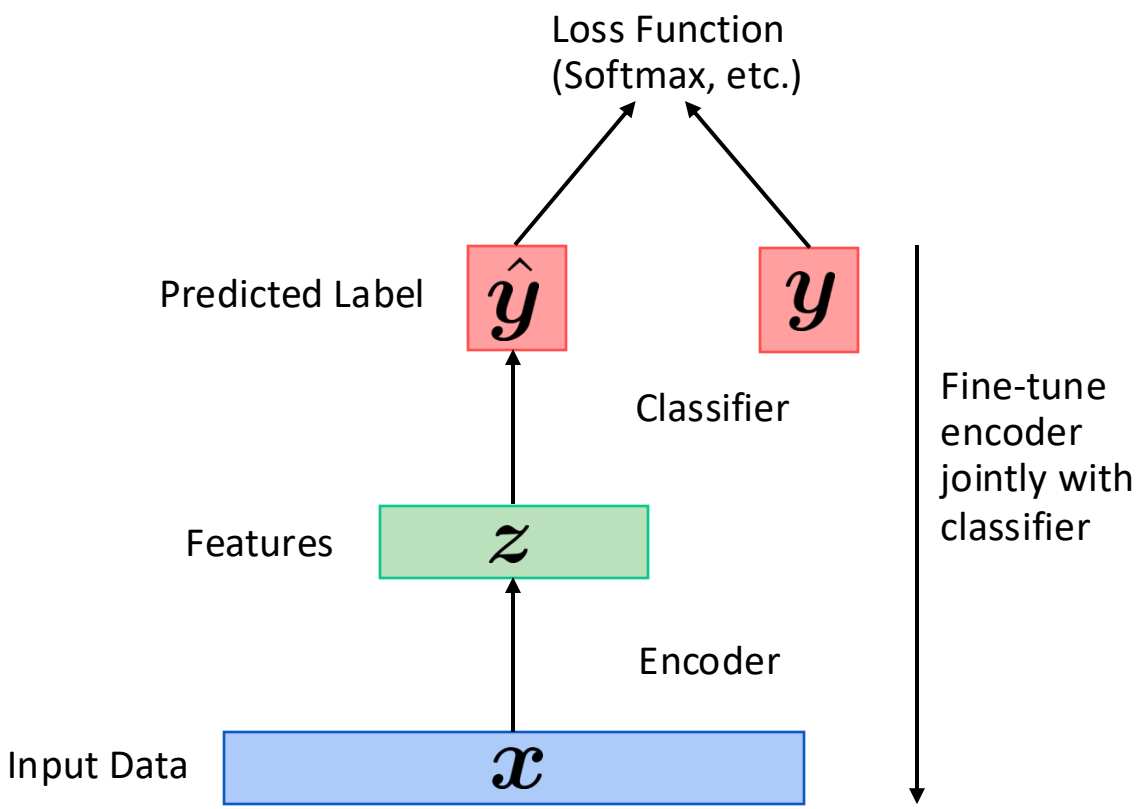
(Regular, non-variational) Autoencoders

- After training, **throw away decoder** and use encoder for a downstream task



(Regular, non-variational) Autoencoders

- After training, **throw away decoder** and use encoder for a downstream task



Encoder can be used to initialize a **supervised** model

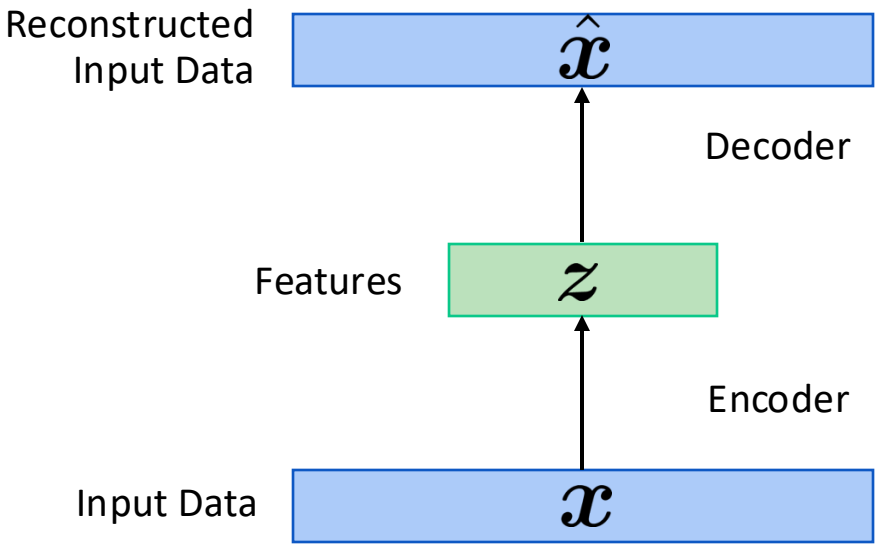
bird plane
dog deer truck

Train for final task

(Sometimes with small data)

(Regular, non-variational) Autoencoders

- Autoencoders learn **latent features** for data without any labels!
- Can use features to initialize a **supervised** model
- Not probabilistic: No way to sample new data from learned model



Variational Autoencoders

Variational Inference

Probabilistic Machine Learning

- A probabilistic model
  - A joint distribution of hidden variables $\boldsymbol{z}$ and observed variables $\boldsymbol{x}$

$$p(\boldsymbol{z}, \boldsymbol{x})$$

  - Describes how (a portion of) the world works

- Inference about the unknowns:
  - Through **posterior** $p(\boldsymbol{z}|\boldsymbol{x})$
  - Conditional distribution of the hidden variables given the observations

$$p(\boldsymbol{z} \mid \boldsymbol{x}) = \frac{p(\boldsymbol{z}, \boldsymbol{x})}{p(\boldsymbol{x})} \begin{array}{l} \leftarrow \text{model} \\ \leftarrow \text{data} \end{array}$$

  - The posterior links data and model ⇒ used in all downstream analyses

- Posterior inference:
  - Therefore, a central task in probabilistic models

Posterior Inference

- Refers to
  - Computing posterior distribution or
  - Taking expectations computed wrt this distribution

e.g. expectation-maximization (EM) algorithm

- Evaluates expectation of complete-data log likelihood wrt posterior distribution of latent variables

$$\mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{\text{old}}) \triangleq \sum_{\mathbf{z}} p(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{\theta}^{\text{old}}) \log p(\boldsymbol{x}, \boldsymbol{z} \mid \boldsymbol{\theta})$$

- For many models of practical interest
  - It is intractable to perform posterior inference ⇒ called "challenge of inference"
- Challenge of inference
  - Makes it difficult to train probabilistic models

Challenge of Inference

- General reasons
  - Dimensionality of latent space: too high to work with directly
  - Posterior: highly complex ⇒ expectations are not analytically tractable

- Reasons in deep learning
  - Interactions between hidden variables (e.g. connections between layers)
  - Most neural networks with multiple layers of hidden variables
    - Have intractable posterior distributions

- A solution: approximate posterior inference

Bayesian view

- We set up the general problem
  - Consider a joint density of latent variables z and observations x

$$p(\boldsymbol{z}, \boldsymbol{x}) = p(\boldsymbol{z})p(\boldsymbol{x} \mid \boldsymbol{z})$$

  - Latent variables help govern distribution of data in Bayesian models

- A Bayesian model
  - Draws latent variables from a prior $p(\boldsymbol{z})$, and then
  - Relates them to observations through likelihood $p(\boldsymbol{x} \mid \boldsymbol{z})$

- Inference in a Bayesian model
  - Conditions on data and computes posterior $p(\boldsymbol{z} \mid \boldsymbol{x})$
  - Computation of posterior often requires approximate inference

Approximate posterior inference

- Stochastic (e.g. Markov chain Monte Carlo: MCMC)
  - Given infinite computational resource, can generate exact results
  - Approximation arises from using a finite amount of processor time

  ☹ sampling methods can be computationally demanding

  ⇒ their use is often limited to small-scale problems

  ☹ difficult to know whether independent samples are being generated

- Deterministic (e.g. variational inference: VI)
  - Some of which scale well to large applications
  - Based on analytical approximations to poster distribution

  e.g. assume it factorizes or has a parametric form (like Gaussian)

  ⇒ never generate exact results

Variational inference (VI)

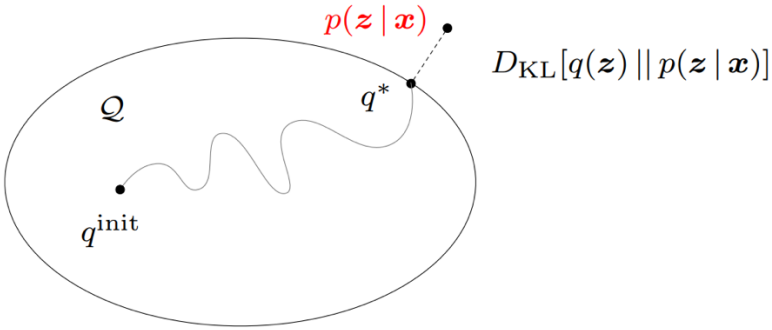
- Main idea: turn inference into optimization (rather than sampling)

- Posit $\mathcal{Q}$, a family of approximate densities over $\boldsymbol{z}$
  - each $q(\boldsymbol{z}) \in \mathcal{Q}$: a candidate approximation to the exact posterior

- Find the member of $\mathcal{Q}$ that minimizes D_{KL} to the exact posterior

$$q^*(\boldsymbol{z}) = \underset{q(\boldsymbol{z}) \in \mathcal{Q}}{\operatorname{argmin}} \, D_{\mathrm{KL}}[q(\boldsymbol{z}) \,||\, p(\boldsymbol{z} \mid \boldsymbol{x})]$$

- Approximate the posterior with q^*
  - Use $q^*(\boldsymbol{z})$ in place of $p(\boldsymbol{z}|\boldsymbol{x})$ in downstream modeling



The phrase "variational"

- An umbrella term that refers to mathematical tools for
  - Optimization-based formulations of problems
  - Associated techniques for their solution

- Origin: calculus of variations (optimization over a space of functions)
- General idea:
  - Express a quantity of interest as the solution of an optimization problem

- The optimization problem can then be relaxed
  - Either by approximating
    - The function to be optimized or
    - The set over which the optimization takes place
  - Such relaxations allow us to
    - Approximate the original quantity of interest

Variational inference (variational Bayes)

- Inference = solving the following quantization:

$$q^*(\boldsymbol{z}) = \underset{q(\boldsymbol{z}) \in \mathcal{Q}}{\operatorname{argmin}} D_{\mathrm{KL}}[q(\boldsymbol{z}) \,||\, p(\boldsymbol{z} \,|\, \boldsymbol{x})] \tag{1}$$

  - Complexity of $\mathcal{Q}$ determines the complexity of this optimization

- Choosing $\mathcal{Q}$ needs a good balance
  - Restrict $\mathcal{Q}$ sufficiently [for efficiency]

    $\Rightarrow$ It comprises only tractable distributions
  - Allow $\mathcal{Q}$ to be sufficiently rich/flexible [for accuracy]

    $\Rightarrow$ it can provide a good approximation to the true posterior

Variational inference (variational Bayes)

- The objective (1): not computable

$$q^*(\boldsymbol{z}) = \underset{q(\boldsymbol{z}) \in \mathcal{Q}}{\operatorname{argmin}} \, D_{\mathrm{KL}}[q(\boldsymbol{z}) \,||\, p(\boldsymbol{z} \,|\, \boldsymbol{x})] \tag{1}$$

- Reason:

$$\begin{aligned} D_{\mathrm{KL}}[q(\boldsymbol{z}) \,||\, p(\boldsymbol{z} \,|\, \boldsymbol{x})] &= \mathbb{E}_{\mathbf{z} \sim q(\boldsymbol{z})}[\log q(\boldsymbol{z})] - \mathbb{E}_{\mathbf{z} \sim q(\boldsymbol{z})}[\log p(\boldsymbol{z} \,|\, \boldsymbol{x})] \\ &= \mathbb{E}_{\mathbf{z}}[\log q(\boldsymbol{z})] - \mathbb{E}_{\mathbf{z}}[\log p(\boldsymbol{z}, \boldsymbol{x})] + \underbrace{\log p(\boldsymbol{x})}_{\text{intractable}} \end{aligned} \tag{2}$$

- Solution:
  - Optimize an alternative objective called evidence lower bound (ELBO)
  - ELBO: aka negative variational free energy

Evidence Lower Bound (ELBO)

- Rearranging (2) gives:

$$\underbrace{\log p(\boldsymbol{x})}_{\substack{\text{a constant}\\ \text{wrt } q(\boldsymbol{z})}} - D_{\mathrm{KL}}[q(\boldsymbol{z}) \,||\, p(\boldsymbol{z} \mid \boldsymbol{x})] = \underbrace{\mathbb{E}_{\mathbf{z}}[\log p(\boldsymbol{z}, \boldsymbol{x})] - \mathbb{E}_{\mathbf{z}}[\log q(\boldsymbol{z})]}_{\triangleq \mathcal{L}(q) \quad \text{“ELBO”}} \tag{3}$$

  - Maximize ELBO = minimize the D_{KL}

- Examining ELBO gives intuitions about optimal variational density

$$\begin{aligned}
\mathcal{L}(q) &= \mathbb{E}_{\mathbf{z}}[\log p(\boldsymbol{z}, \boldsymbol{x})] - \mathbb{E}_{\mathbf{z}}[\log q(\boldsymbol{z})] \\
&= \mathbb{E}_{\mathbf{z}}[\log p(\boldsymbol{z})] + \mathbb{E}_{\mathbf{z}}[\log p(\boldsymbol{x} \mid \boldsymbol{z})] - \mathbb{E}_{\mathbf{z}}[\log q(\boldsymbol{z})] \\
&= \underbrace{\mathbb{E}_{\mathbf{z}}[\log p(\boldsymbol{x} \mid \boldsymbol{z})]}_{\substack{\text{expected log likelihood}\\ \text{of the data}}} - \underbrace{D_{\mathrm{KL}}[q(\boldsymbol{z}) \,||\, p(\boldsymbol{z})]}_{\substack{D_{\mathrm{KL}} \text{ between}\\ \text{prior } p(\boldsymbol{z}) \text{ and } q(\boldsymbol{z})}}
\end{aligned}$$

Evidence Lower Bound (ELBO)

ELBO ("variational objective"): $\mathcal{L}(q) = \mathbb{E}_{\mathbf{z}\sim q(\boldsymbol{z})}[\log p(\boldsymbol{x} \mid \boldsymbol{z})] - D_{\mathrm{KL}}[q(\boldsymbol{z}) \,||\, p(\boldsymbol{z})]$

Which values of $\boldsymbol{z}$ will this objective encourage $q(\boldsymbol{z})$ to place its mass on?

- First term: $\mathbb{E}_{\mathbf{z}}[\log p(\boldsymbol{x} \mid \boldsymbol{z})]$

  i.e. an expected likelihood

  - Encourages densities that explain observed data

- Second term: $D_{\mathrm{KL}}[q(\boldsymbol{z}) \,||\, p(\boldsymbol{z})]$

  i.e. negative divergence between variational density and prior

  - Encourages densities close to prior

- Thus, the variational objective mirrors

  - The usual balance between likelihood and prior

Why called ELBO?

- ELBO $\mathcal{L}(q)$ lower-bounds the evidence $p(\boldsymbol{x})$

  i.e. for any $q(\boldsymbol{z})$

$$\log p(\boldsymbol{x}) \geq \mathcal{L}(q)$$

- proof:
  - Notice that (3) gives

$$\log p(\boldsymbol{x}) = \mathcal{L}(q) + \underbrace{D_{\mathrm{KL}}[q(\boldsymbol{z}) \,||\, p(\boldsymbol{z} \mid \boldsymbol{x})]}_{\geq 0}$$

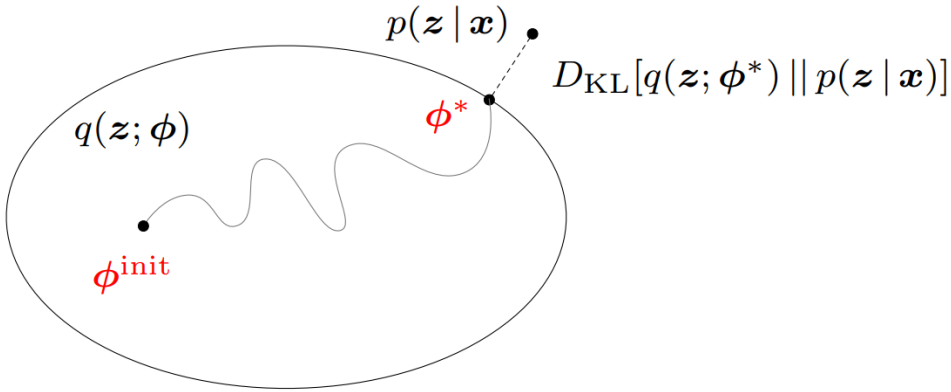
  - Can also be derived through Jensen's inequality

Common restrictions on $\mathcal{Q}$

- parameterization
  - Use a parametric distribution governed by parameters $\boldsymbol{\phi}$

$$q(\boldsymbol{z};\boldsymbol{\phi}) = q_{\boldsymbol{\phi}}(\boldsymbol{z})$$

  - $\boldsymbol{\phi}$: variational parameters



  - ELBO $\mathcal{L}$ now becomes a function of $\boldsymbol{\phi}$: $\mathcal{L}(q,\boldsymbol{\phi})$
  - Inference = finding $\boldsymbol{\phi}^*$
  - Spoiler: VAE uses a neural network to model $q_{\boldsymbol{\phi}}(\boldsymbol{z})$ and sgd to train it

Common restrictions on $\mathcal{Q}$

- factorization
  - Assume q factorizes (called "mean-field approach")

$$q(\boldsymbol{z}) = \prod_{j=1}^{m} q_j(z_j)$$

  - Each latent variable z_j:
    - Governed by its own variational factor, the density $q_j(z_j)$
    - Can take on any form appropriate to the corresponding random variable

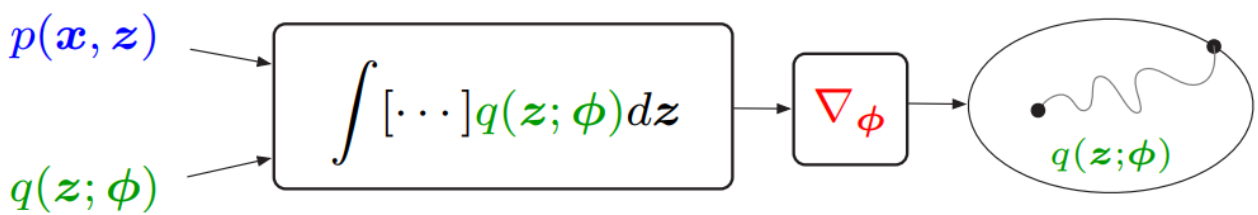
    e.g. a continuous variable: a Gaussian factor

    a categorical variable: a categorical factor

core idea behind VI: maximize $\mathcal{L}$ over a restricted family of distributions q

Variational inference recipe

1. start with a model: $p(\boldsymbol{x}, \boldsymbol{z})$
2. choose a variational approximation: $q(\boldsymbol{z}; \boldsymbol{\phi}) = q_{\boldsymbol{\phi}}(\boldsymbol{z})$
3. write down ELBO: $\mathcal{L}(\boldsymbol{\phi}) = \mathbb{E}_{\mathbf{z} \sim q(\mathbf{z}; \boldsymbol{\phi})}[\log p(\boldsymbol{x}, \boldsymbol{z}) - \log q(\boldsymbol{z}; \boldsymbol{\phi})]$
4. compute the expectation/integral: *e.g.* $\mathcal{L}(\boldsymbol{\phi}) = \boldsymbol{x}\boldsymbol{\phi}^2 + \log \boldsymbol{\phi}$
5. take derivatives: *e.g.* $\nabla_{\boldsymbol{\phi}} \mathcal{L}(\boldsymbol{\phi}) = 2\boldsymbol{x}\boldsymbol{\phi} + \frac{1}{\boldsymbol{\phi}}$
6. optimize: $\boldsymbol{\phi}_{t+1} = \boldsymbol{\phi}_t + \epsilon \nabla_{\boldsymbol{\phi}} \mathcal{L}(\boldsymbol{\phi})$



Variational Inference

Variational Autoencoders

Variational Autoencoders

Probabilistic spin on autoencoders:

- Learn latent features **z** from raw data
- Sample from the model to generate new data

Variational Autoencoders

Probabilistic spin on autoencoders:

- Learn latent features **z** from raw data
- Sample from the model to generate new data

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

Intuition: x is an image, **z** is latent factors used to generate **x**:
e.g., attributes, orientation, etc.

Variational Autoencoders

Probabilistic spin on autoencoders:

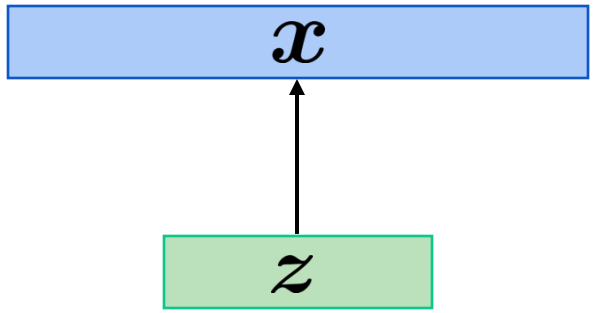
- Learn latent features **z** from raw data
- Sample from the model to generate new data

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

Intuition: x is an image, **z** is latent factors used to generate **x**:
e.g., attributes, orientation, etc.

After training, sample new data as follows:

Sample from conditional $p_{\boldsymbol{\theta}^*}(\boldsymbol{x}|\boldsymbol{z}^{(i)})$



Sample **z** from prior $p_{\boldsymbol{\theta}^*}(\boldsymbol{z})$

Variational Autoencoders

Probabilistic spin on autoencoders:

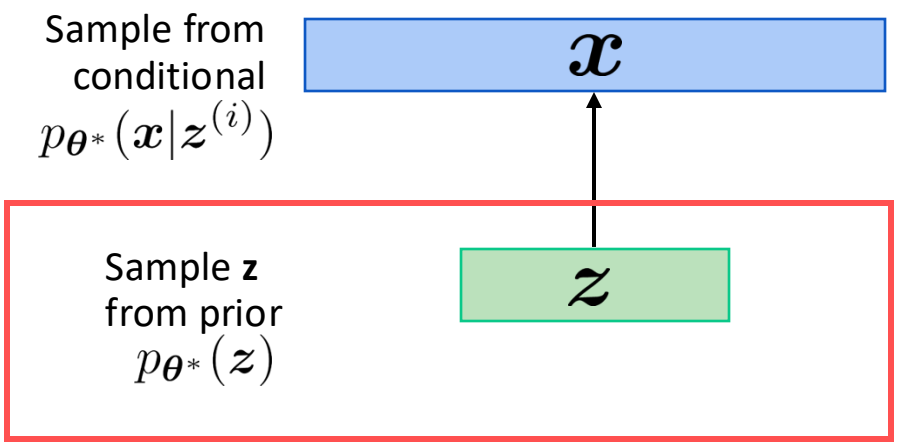
- Learn latent features **z** from raw data
- Sample from the model to generate new data

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

Intuition: x is an image, **z** is latent factors used to generate **x**:
e.g., attributes, orientation, etc.

After training, sample new data as follows:

Assume simple prior, e.g., Gaussian



Variational Autoencoders

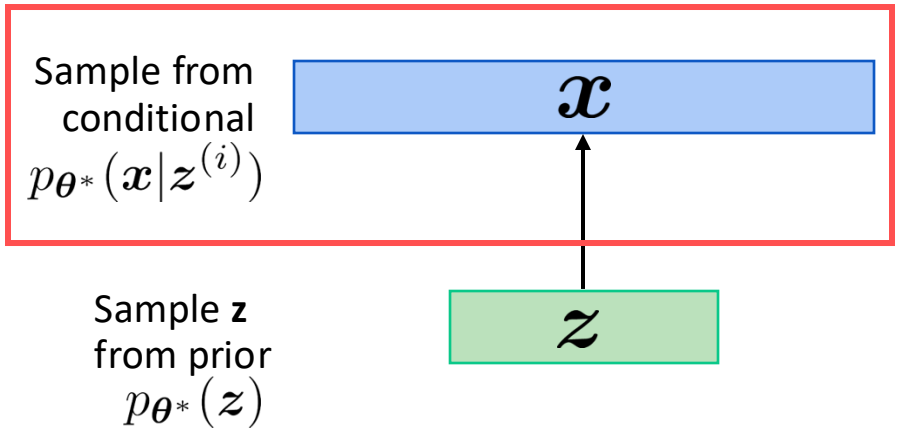
Probabilistic spin on autoencoders:

- Learn latent features **z** from raw data
- Sample from the model to generate new data

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

Intuition: x is an image, **z** is latent factors used to generate **x**:
e.g., attributes, orientation, etc.

After training, sample new data as follows:



Assume simple prior, e.g., Gaussian

Represent conditional with a neural network (similar to **decoder** from autoencoder)

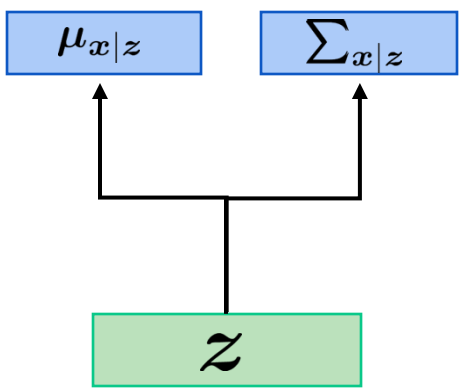
Variational Autoencoders

Decoder must be **probabilistic**:

Decoder takes **z** as input and produces mean $\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}$ and (diagonal) covariance $\sum_{\boldsymbol{x}|\boldsymbol{z}}$ as output

Sample x from Gaussian with mean $\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}$ and (diagonal) covariance $\sum_{\boldsymbol{x}|\boldsymbol{z}}$

Sample from conditional $p_{\boldsymbol{\theta}^*}(\boldsymbol{x}|\boldsymbol{z}^{(i)})$



Sample **z** from prior $p_{\boldsymbol{\theta}^*}(\boldsymbol{z})$

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

Intuition: x is an image, **z** is latent factors used to generate **x**:
e.g., attributes, orientation, etc.

Assume simple prior, e.g., Gaussian

Represent conditional with a neural network (similar to **decoder** from autoencoder)

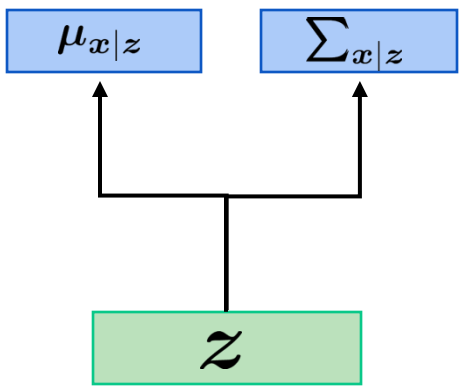
Variational Autoencoders

Decoder must be **probabilistic**:

Decoder takes **z** as input and produces mean $\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}$ and (diagonal) covariance $\sum_{\boldsymbol{x}|\boldsymbol{z}}$ as output

Sample x from Gaussian with mean $\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}$ and (diagonal) covariance $\sum_{\boldsymbol{x}|\boldsymbol{z}}$

Sample from conditional $p_{\boldsymbol{\theta}^*}(\boldsymbol{x}|\boldsymbol{z}^{(i)})$



Sample **z** from prior $p_{\boldsymbol{\theta}^*}(\boldsymbol{z})$

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

Intuition: x is an image, **z** is latent factors used to generate **x**:
e.g., attributes, orientation, etc.

How to train this model?

Basic idea: **maximize likelihood of data**

If we could observe $\boldsymbol{z}$ for each $\boldsymbol{x}$, then could train a conditional generative model $p(\boldsymbol{x}|\boldsymbol{z})$

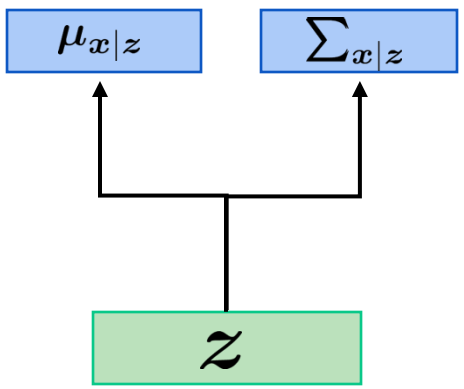
Variational Autoencoders

Decoder must be **probabilistic**:

Decoder takes **z** as input and produces mean $\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}$ and (diagonal) covariance $\sum_{\boldsymbol{x}|\boldsymbol{z}}$ as output

Sample x from Gaussian with mean $\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}$ and (diagonal) covariance $\sum_{\boldsymbol{x}|\boldsymbol{z}}$

Sample from conditional $p_{\boldsymbol{\theta}^*}(\boldsymbol{x}|\boldsymbol{z}^{(i)})$



Sample **z** from prior $p_{\boldsymbol{\theta}^*}(\boldsymbol{z})$

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

Intuition: x is an image, **z** is latent factors used to generate **x**:
e.g., attributes, orientation, etc.

How to train this model?

Basic idea: **maximize likelihood of data**

We don't observe z, so need to marginalize

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \int p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z}) d\boldsymbol{z} = \int p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}) p_{\boldsymbol{\theta}}(\boldsymbol{z}) d\boldsymbol{z}$$

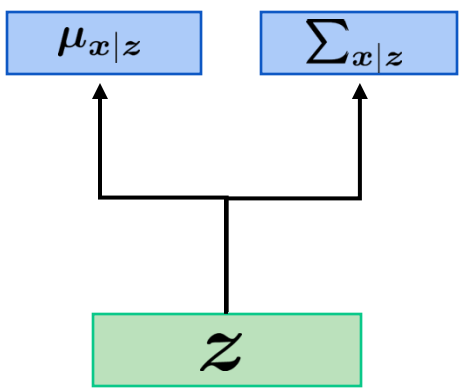
Variational Autoencoders

Decoder must be **probabilistic**:

Decoder takes **z** as input and produces mean $\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}$ and (diagonal) covariance $\sum_{\boldsymbol{x}|\boldsymbol{z}}$ as output

Sample x from Gaussian with mean $\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}$ and (diagonal) covariance $\sum_{\boldsymbol{x}|\boldsymbol{z}}$

Sample from conditional $p_{\boldsymbol{\theta}^*}(\boldsymbol{x}|\boldsymbol{z}^{(i)})$



Sample **z** from prior $p_{\boldsymbol{\theta}^*}(\boldsymbol{z})$

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

Intuition: x is an image, **z** is latent factors used to generate **x**: e.g., attributes, orientation, etc.

How to train this model?

Basic idea: **maximize likelihood of data**

We don't observe z, so need to marginalize

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \int p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z}) d\boldsymbol{z} = \int \boxed{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})} p_{\boldsymbol{\theta}}(\boldsymbol{z}) d\boldsymbol{z}$$

Ok, can compute this with a decoder network

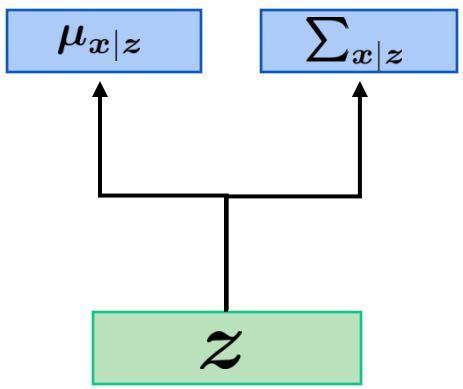
Variational Autoencoders

Decoder must be **probabilistic**:

Decoder takes **z** as input and produces mean $\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}$ and (diagonal) covariance $\sum_{\boldsymbol{x}|\boldsymbol{z}}$ as output

Sample x from Gaussian with mean $\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}$ and (diagonal) covariance $\sum_{\boldsymbol{x}|\boldsymbol{z}}$

Sample from conditional $p_{\boldsymbol{\theta}^*}(\boldsymbol{x}|\boldsymbol{z}^{(i)})$



Sample **z** from prior $p_{\boldsymbol{\theta}^*}(\boldsymbol{z})$

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

Intuition: x is an image, **z** is latent factors used to generate **x**: e.g., attributes, orientation, etc.

How to train this model?

Basic idea: **maximize likelihood of data**

We don't observe z, so need to marginalize

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \int p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z}) d\boldsymbol{z} = \int p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}) \boxed{p_{\boldsymbol{\theta}}(\boldsymbol{z})} d\boldsymbol{z}$$

Ok, we assumed Gaussian prior for $\boldsymbol{z}$

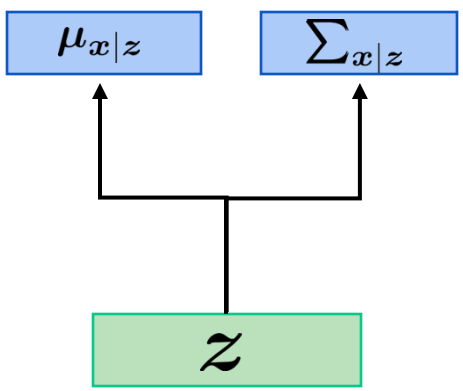
Variational Autoencoders

Decoder must be **probabilistic**:

Decoder takes **z** as input and produces mean $\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}$ and (diagonal) covariance $\sum_{\boldsymbol{x}|\boldsymbol{z}}$ as output

Sample x from Gaussian with mean $\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}$ and (diagonal) covariance $\sum_{\boldsymbol{x}|\boldsymbol{z}}$

Sample from conditional $p_{\boldsymbol{\theta}^*}(\boldsymbol{x}|\boldsymbol{z}^{(i)})$



Sample **z** from prior $p_{\boldsymbol{\theta}^*}(\boldsymbol{z})$

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

Intuition: x is an image, **z** is latent factors used to generate **x**: e.g., attributes, orientation, etc.

How to train this model?

Basic idea: **maximize likelihood of data**

We don't observe z, so need to marginalize

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \int p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z}) d\boldsymbol{z} = \boxed{\int} p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}) p_{\boldsymbol{\theta}}(\boldsymbol{z}) \boxed{d\boldsymbol{z}}$$

Problem: impossible to integrate over all $\boldsymbol{z}$!

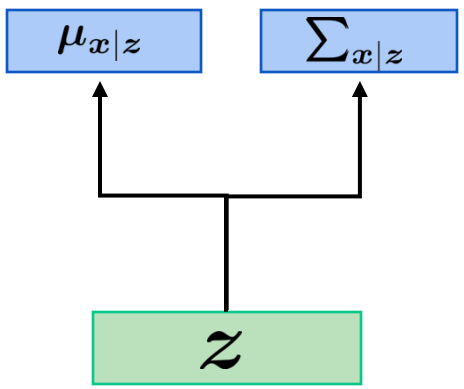
Variational Autoencoders

Decoder must be **probabilistic**:

Decoder takes **z** as input and produces mean $\boldsymbol{\mu}_{x|z}$ and (diagonal) covariance $\sum_{x|z}$ as output

Sample x from Gaussian with mean $\boldsymbol{\mu}_{x|z}$ and (diagonal) covariance $\sum_{x|z}$

Sample from conditional $p_{\theta^*}(\boldsymbol{x}|\boldsymbol{z}^{(i)})$



Sample **z** from prior $p_{\theta^*}(\boldsymbol{z})$

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

Intuition: x is an image, **z** is latent factors used to generate **x**: e.g., attributes, orientation, etc.

How to train this model?

Basic idea: **maximize likelihood of data**

Another Idea: Try Bayes' Rule

$$p_{\theta}(\boldsymbol{x}) = \frac{p_{\theta}(\boldsymbol{x}|\boldsymbol{z})p_{\theta}(\boldsymbol{z})}{p_{\theta}(\boldsymbol{z}|\boldsymbol{x})}$$

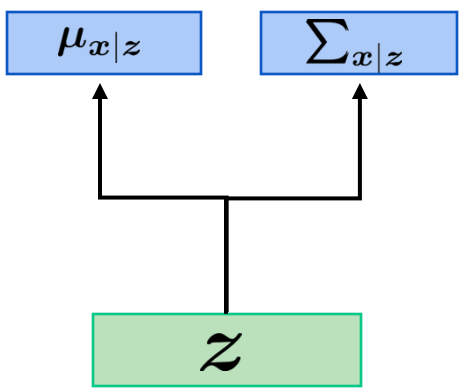
Variational Autoencoders

Decoder must be **probabilistic**:

Decoder takes **z** as input and produces mean $\boldsymbol{\mu}_{x|z}$ and (diagonal) covariance $\sum_{x|z}$ as output

Sample x from Gaussian with mean $\boldsymbol{\mu}_{x|z}$ and (diagonal) covariance $\sum_{x|z}$

Sample from conditional $p_{\theta^*}(\boldsymbol{x}|\boldsymbol{z}^{(i)})$



Sample **z** from prior $p_{\theta^*}(\boldsymbol{z})$

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

Intuition: x is an image, **z** is latent factors used to generate **x**: e.g., attributes, orientation, etc.

How to train this model?

Basic idea: **maximize likelihood of data**

Another Idea: Try Bayes' Rule

$$p_{\theta}(\boldsymbol{x}) = \frac{\boxed{p_{\theta}(\boldsymbol{x}|\boldsymbol{z})}p_{\theta}(\boldsymbol{z})}{p_{\theta}(\boldsymbol{z}|\boldsymbol{x})}$$

Ok, can compute this with a decoder network

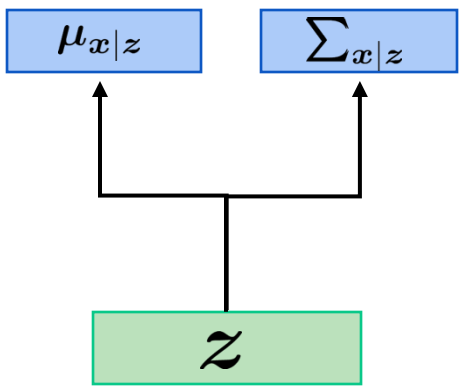
Variational Autoencoders

Decoder must be **probabilistic**:

Decoder takes **z** as input and produces mean $\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}$ and (diagonal) covariance $\sum_{\boldsymbol{x}|\boldsymbol{z}}$ as output

Sample x from Gaussian with mean $\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}$ and (diagonal) covariance $\sum_{\boldsymbol{x}|\boldsymbol{z}}$

Sample from conditional $p_{\boldsymbol{\theta}^*}(\boldsymbol{x}|\boldsymbol{z}^{(i)})$



Sample **z** from prior $p_{\boldsymbol{\theta}^*}(\boldsymbol{z})$

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

Intuition: x is an image, **z** is latent factors used to generate **x**: e.g., attributes, orientation, etc.

How to train this model?

Basic idea: **maximize likelihood of data**

Another Idea: Try Bayes' Rule

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\boxed{p_{\boldsymbol{\theta}}(\boldsymbol{z})}}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})}$$

Ok, we assumed Gaussian prior

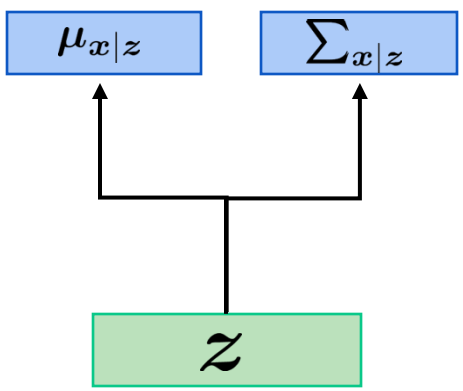
Variational Autoencoders

Decoder must be **probabilistic**:

Decoder takes **z** as input and produces mean $\boldsymbol{\mu}_{x|z}$ and (diagonal) covariance $\sum_{x|z}$ as output

Sample x from Gaussian with mean $\boldsymbol{\mu}_{x|z}$ and (diagonal) covariance $\sum_{x|z}$

Sample from conditional $p_{\theta^*}(\boldsymbol{x}|\boldsymbol{z}^{(i)})$



Sample **z** from prior $p_{\theta^*}(\boldsymbol{z})$

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

Intuition: x is an image, **z** is latent factors used to generate **x**: e.g., attributes, orientation, etc.

How to train this model?

Basic idea: **maximize likelihood of data**

Another Idea: Try Bayes' Rule

$$p_{\theta}(\boldsymbol{x}) = \frac{p_{\theta}(\boldsymbol{x}|\boldsymbol{z})p_{\theta}(\boldsymbol{z})}{\boxed{p_{\theta}(\boldsymbol{z}|\boldsymbol{x})}}$$

Problem: No way to compute this!

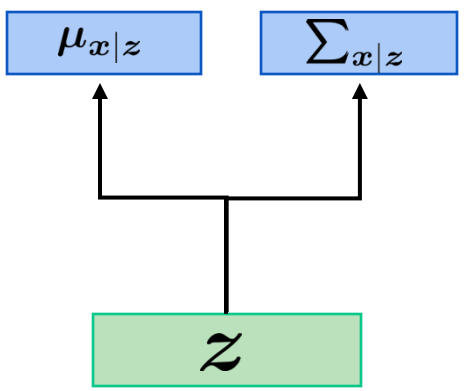
Variational Autoencoders

Decoder must be **probabilistic**:

Decoder takes **z** as input and produces mean $\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}$ and (diagonal) covariance $\sum_{\boldsymbol{x}|\boldsymbol{z}}$ as output

Sample x from Gaussian with mean $\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}$ and (diagonal) covariance $\sum_{\boldsymbol{x}|\boldsymbol{z}}$

Sample from conditional $p_{\boldsymbol{\theta}^*}(\boldsymbol{x}|\boldsymbol{z}^{(i)})$



Sample **z** from prior $p_{\boldsymbol{\theta}^*}(\boldsymbol{z})$

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

Intuition: x is an image, **z** is latent factors used to generate **x**: e.g., attributes, orientation, etc.

How to train this model?

Basic idea: **maximize likelihood of data**

Another Idea: Try Bayes' Rule

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p_{\boldsymbol{\theta}}(\boldsymbol{z})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})} \approx \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p_{\boldsymbol{\theta}}(\boldsymbol{z})}{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}$$

Solution: train another network (encoder) that learns $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \approx p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$

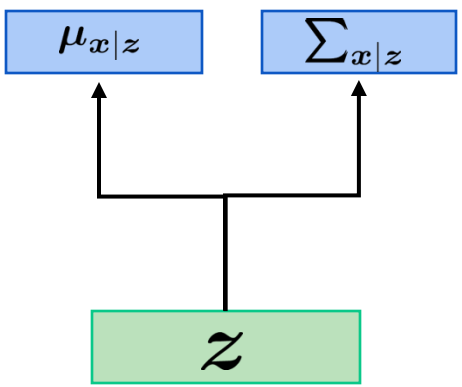
Variational Autoencoders

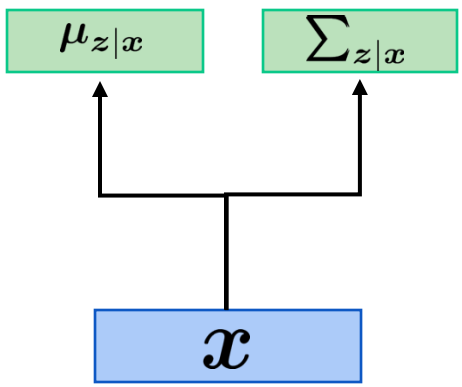
Decoder network takes latent code **z** as input and gives distribution over data **x**

Encoder network takes data **x** as input and gives distribution over latent codes **z**

$$p_\theta(\boldsymbol{x}|\boldsymbol{z}) = \mathcal{N}(\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}, \textstyle\sum_{\boldsymbol{x}|\boldsymbol{z}})$$

$$q_\phi(\boldsymbol{z}|\boldsymbol{x}) = \mathcal{N}(\boldsymbol{\mu}_{\boldsymbol{z}|\boldsymbol{x}}, \textstyle\sum_{\boldsymbol{z}|\boldsymbol{x}})$$





If we can ensure that

$$q_\phi(\boldsymbol{z}|\boldsymbol{x}) \approx p_\theta(\boldsymbol{z}|\boldsymbol{x})$$

Then, we can approximate

$$p_\theta(\boldsymbol{x}) \approx \frac{p_\theta(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{q_\phi(\boldsymbol{z}|\boldsymbol{x})}$$

Idea: jointly train both encoder and decoder

Variational Autoencoders

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})}$$

Bayes' Rule

Variational Autoencoders

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})} = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}$$

Multiply top and bottom by $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$

Variational Autoencoders

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})} = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}$$

$$= \log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}) - \log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p(\boldsymbol{z})} + \log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})}$$

Split up using rule for logarithms

Variational Autoencoders

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})} = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}$$

$$= \log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}) - \log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p(\boldsymbol{z})} + \log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})}$$

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}[\log p_{\boldsymbol{\theta}}(\boldsymbol{x})]$$

We can wrap in an expectation since it doesn't depend on **z**

Variational Autoencoders

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})} = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}$$

$$= \mathbb{E}_{\boldsymbol{z}}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - \mathbb{E}_{\boldsymbol{z}}\left[\log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p(\boldsymbol{z})}\right] + \mathbb{E}_{\boldsymbol{z}}\left[\log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})}\right]$$

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}[\log p_{\boldsymbol{\theta}}(\boldsymbol{x})]$$

We can wrap in an expectation since it doesn't depend on **z**

Variational Autoencoders

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})} = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}$$

$$= \mathbb{E}_{\boldsymbol{z}}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - \mathbb{E}_{\boldsymbol{z}}\left[\log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p(\boldsymbol{z})}\right] + \mathbb{E}_{\boldsymbol{z}}\left[\log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})}\right]$$

$$= \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p(\boldsymbol{z})\right) + D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\right)$$

$$D_{\mathrm{KL}}(P \parallel Q) = \int p(x) \log\left(\frac{p(x)}{q(x)}\right) dx$$

KL-divergence (also called relative entropy): computes a distance between two distributions

Two important properties:

$$D_{\mathrm{KL}}(P \parallel Q) \neq D_{\mathrm{KL}}(Q \parallel P)$$

$$D_{\mathrm{KL}}(P \parallel Q) \geq 0$$

Variational Autoencoders

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})} = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}$$

$$= \mathbb{E}_{\boldsymbol{z}}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - \mathbb{E}_{\boldsymbol{z}}\left[\log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p(\boldsymbol{z})}\right] + \mathbb{E}_{\boldsymbol{z}}\left[\log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})}\right]$$

$$= \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p(\boldsymbol{z})\right) + D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\right)$$

Data reconstruction

Variational Autoencoders

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})} = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}$$

$$= \mathbb{E}_{\boldsymbol{z}}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - \mathbb{E}_{\boldsymbol{z}}\left[\log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p(\boldsymbol{z})}\right] + \mathbb{E}_{\boldsymbol{z}}\left[\log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})}\right]$$

$$= \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p(\boldsymbol{z})\right) + D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\right)$$

KL divergence between prior and samples from the encoder network

Variational Autoencoders

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})} = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}$$

$$= \mathbb{E}_{\boldsymbol{z}}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - \mathbb{E}_{\boldsymbol{z}}\left[\log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p(\boldsymbol{z})}\right] + \mathbb{E}_{\boldsymbol{z}}\left[\log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})}\right]$$

$$= \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p(\boldsymbol{z})\right) + D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\right)$$

KL divergence between encoder and posterior of decoder

Recap: the posterior of decoder is intractable

Variational Autoencoders

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})} = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}$$

$$= \mathbb{E}_{\boldsymbol{z}}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - \mathbb{E}_{\boldsymbol{z}}\left[\log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p(\boldsymbol{z})}\right] + \mathbb{E}_{\boldsymbol{z}}\left[\log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})}\right]$$

$$= \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p(\boldsymbol{z})\right) + D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\right)$$

KL is >= 0, so dropping this term gives a **lower bound** on the data likelihood

Variational Autoencoders

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})} = \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}$$

$$= \mathbb{E}_{\boldsymbol{z}}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - \mathbb{E}_{\boldsymbol{z}}\left[\log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p(\boldsymbol{z})}\right] + \mathbb{E}_{\boldsymbol{z}}\left[\log \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})}\right]$$

$$= \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p(\boldsymbol{z})\right) + D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\right)$$

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) \geq \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p(\boldsymbol{z})\right)$$

Variational Autoencoders

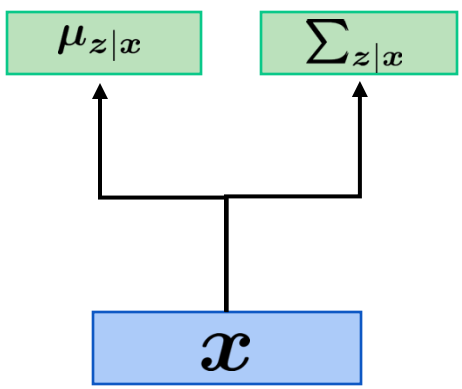
Jointly train **encoder** q and **decoder** p

to maximize the variational lower bound on the data likelihood

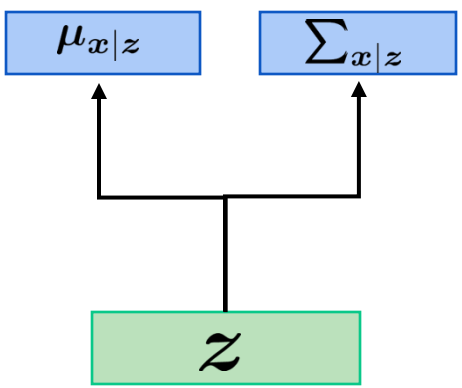
Also called **Evidence Lower Bound (ELBO)**

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) \geq \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p(\boldsymbol{z})\right)$$

Encoder network



Decoder network



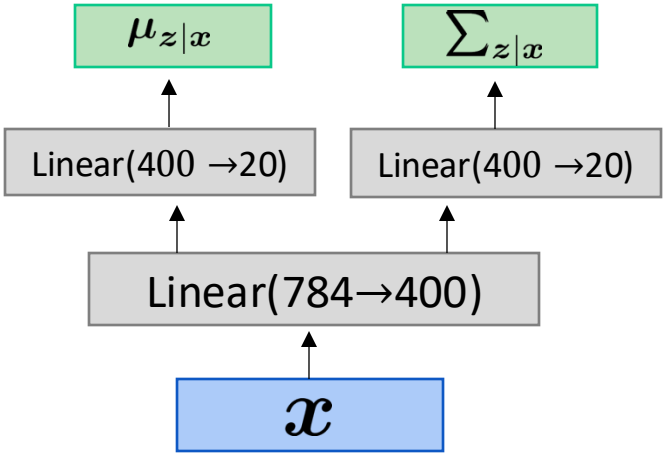
Example: Fully-Connected VAE

x: 28x28 image, flattened to 784-dim vector

z: 20-dim vector

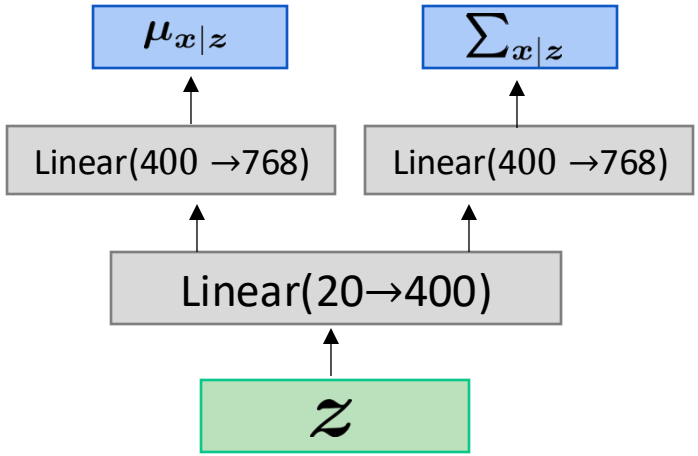
Encoder network

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}) = \mathcal{N}(\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{z}}, \textstyle\sum_{\boldsymbol{x}|\boldsymbol{z}})$$



Decoder network

$$q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) = \mathcal{N}(\boldsymbol{\mu}_{\boldsymbol{z}|\boldsymbol{x}}, \textstyle\sum_{\boldsymbol{z}|\boldsymbol{x}})$$



Variational Autoencoders

Train by maximizing the **variational lower bound**

$$\mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p(\boldsymbol{z})\right)$$

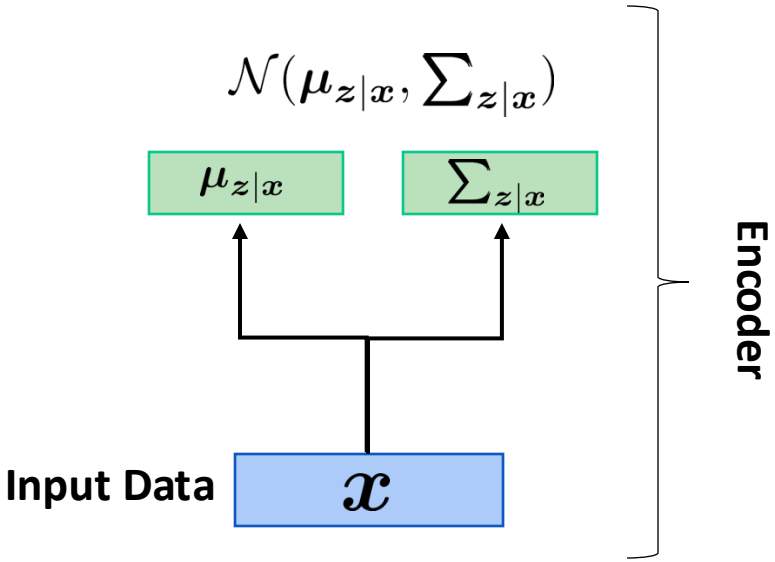
Input Data $\boldsymbol{x}$

Variational Autoencoders

Train by maximizing the **variational lower bound**

$$\mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p(\boldsymbol{z})\right)$$

1. Run input data through **encoder** to get a distribution over latent codes



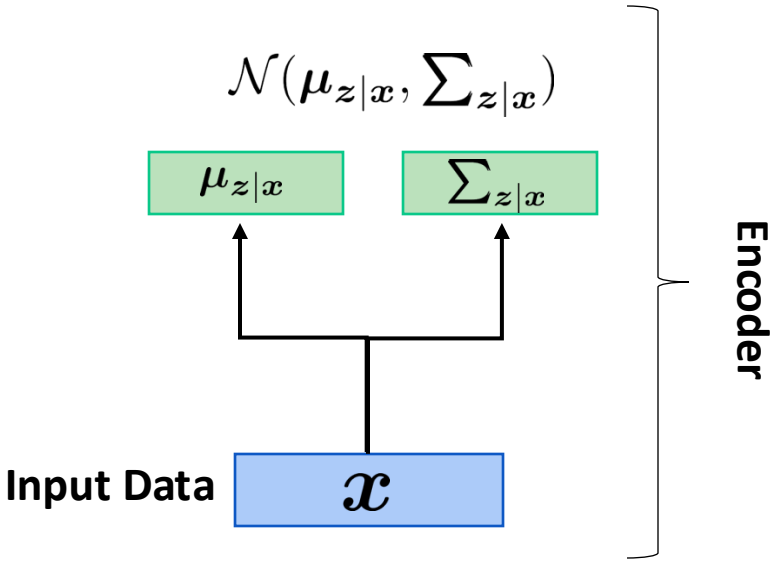
Variational Autoencoders

Train by maximizing the **variational lower bound**

$$\mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - \boxed{D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p(\boldsymbol{z})\right)}$$

Closed form solution when q_ϕ is diagonal Gaussian and p is unit Gaussian!

1. Run input data through **encoder** to get a distribution over latent codes
2. **Encoder output should match the prior $p(z)$!**

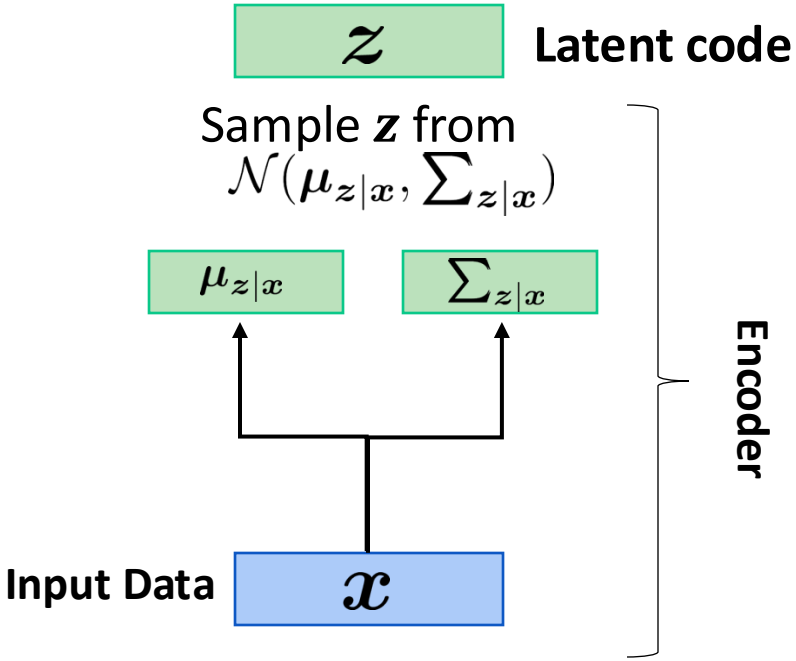


Variational Autoencoders

Train by maximizing the **variational lower bound**

$$\mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - \boxed{D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p(\boldsymbol{z})\right)}$$

1. Run input data through **encoder** to get a distribution over latent codes
2. **Encoder output should match the prior $p(z)$!**
3. Sample code $\boldsymbol{z}$ from encoder output

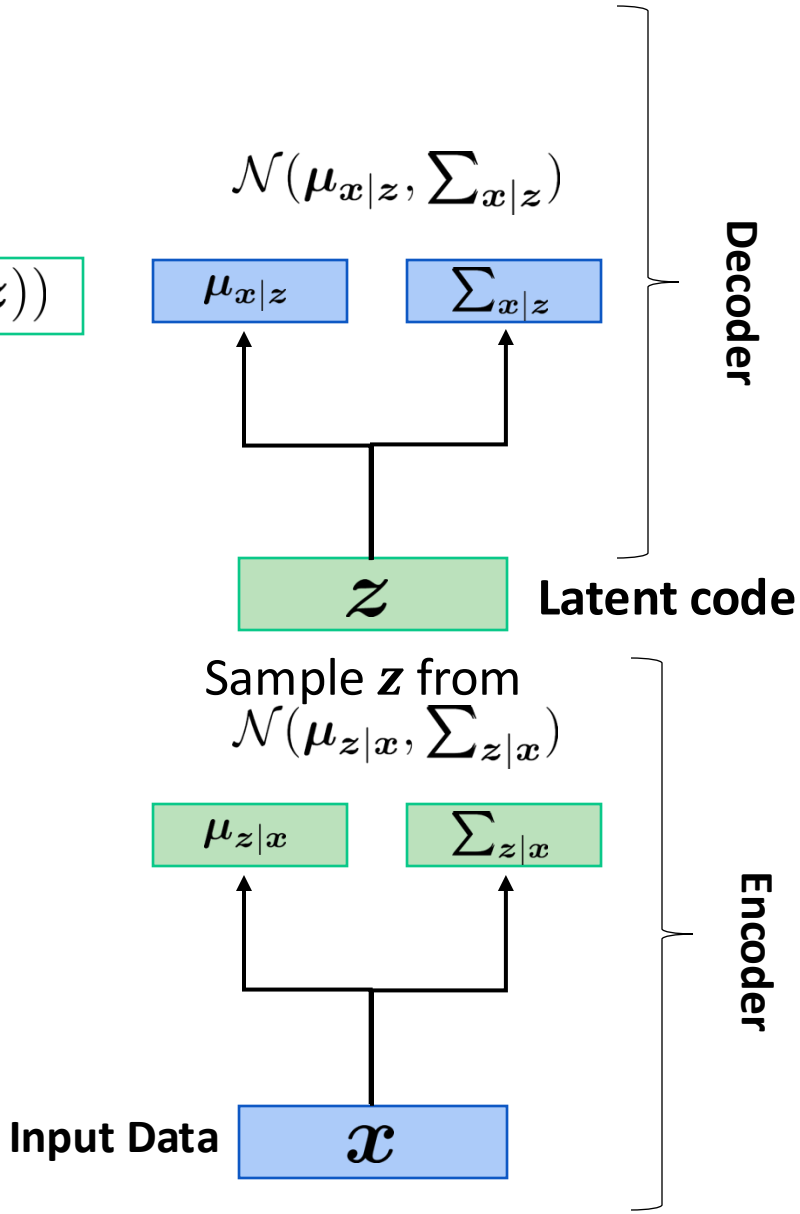


Variational Autoencoders

Train by maximizing the **variational lower bound**

$$\mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - \boxed{D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p(\boldsymbol{z})\right)}$$

1. Run input data through **encoder** to get a distribution over latent codes
2. **Encoder output should match the prior $p(z)$!**
3. Sample code $\boldsymbol{z}$ from encoder output
4. Run sampled code through **decoder** to get a distribution over data samples

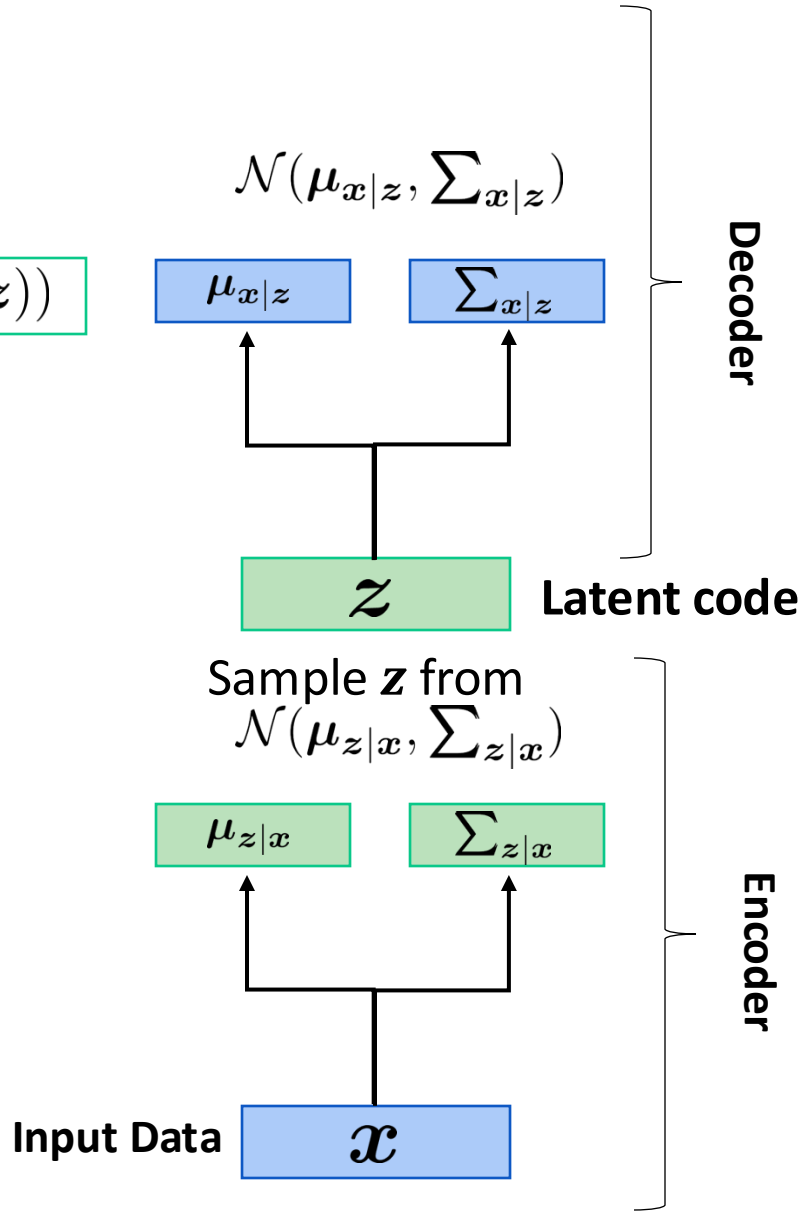


Variational Autoencoders

Train by maximizing the **variational lower bound**

$$\mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p(\boldsymbol{z})\right)$$

1. Run input data through **encoder** to get a distribution over latent codes
2. **Encoder output should match the prior $p(z)$!**
3. Sample code $\boldsymbol{z}$ from encoder output
4. Run sampled code through **decoder** to get a distribution over data samples
5. **Original input data should be likely under the distribution output from step 4**

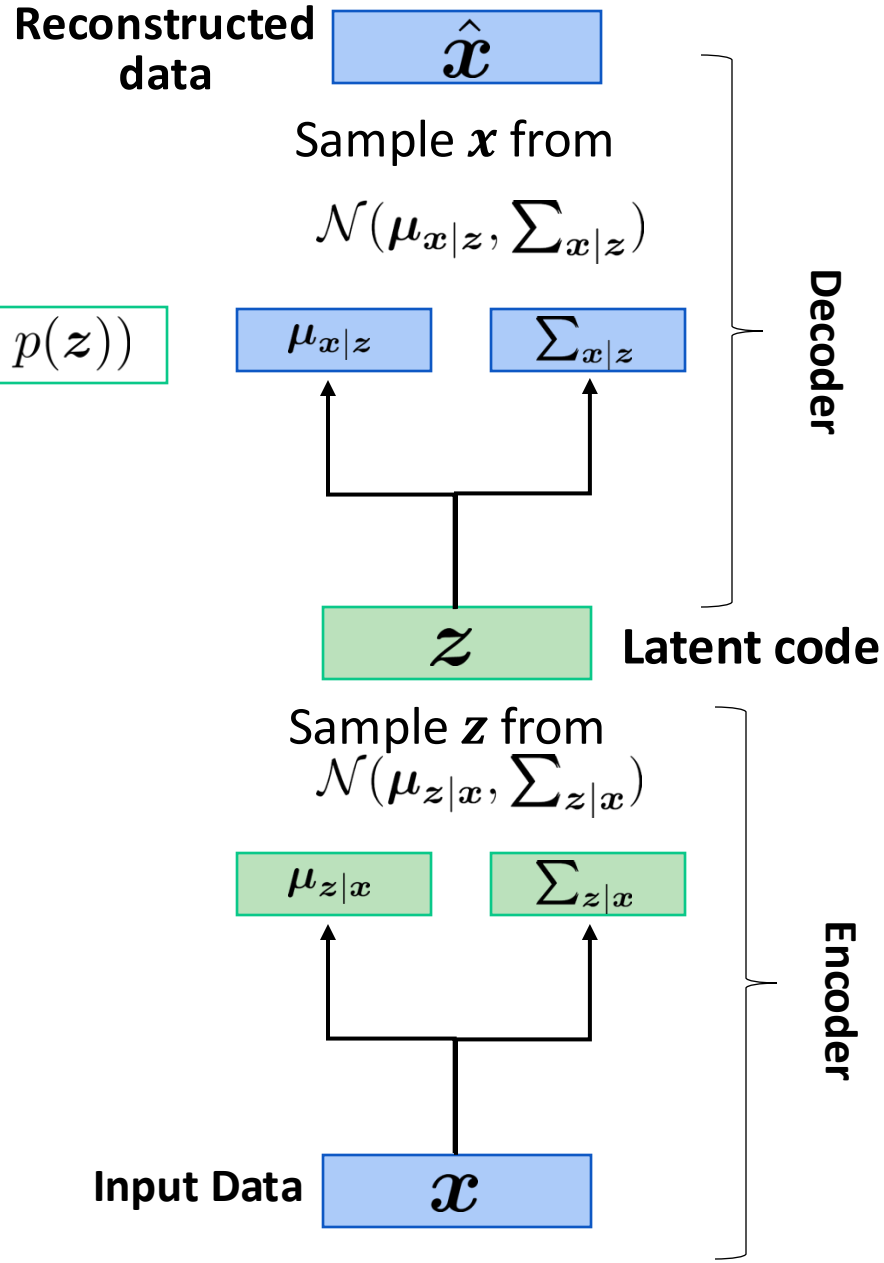


Variational Autoencoders

Train by maximizing the **variational lower bound**

$$\mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})\right] - D_{\mathrm{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p(\boldsymbol{z})\right)$$

1. Run input data through **encoder** to get a distribution over latent codes
2. **Encoder output should match the prior $p(z)$!**
3. Sample code $\boldsymbol{z}$ from encoder output
4. Run sampled code through **decoder** to get a distribution over data samples
5. **Original input data should be likely under the distribution output from step 4**
6. Can sample a reconstruction from step 4



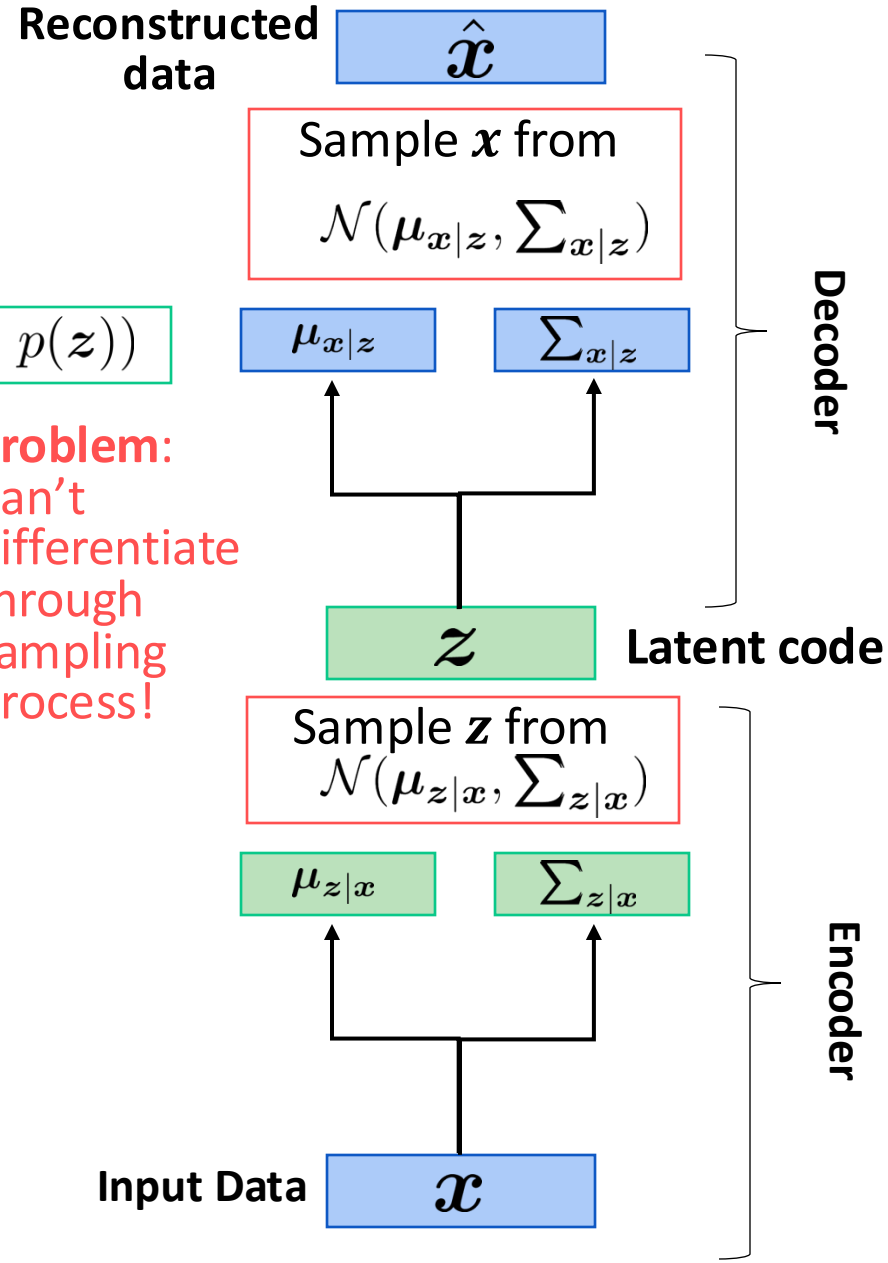
Variational Autoencoders

Train by maximizing the **variational lower bound**

$$\mathbb{E}_{z\sim q_\phi(z|x)}\left[\log p_\theta(x|z)\right] - D_{\mathrm{KL}}\left(q_\phi(z|x) \parallel p(z)\right)$$

1. Run input data through **encoder** to get a distribution over latent codes
2. **Encoder output should match the prior $p(z)$!**
3. Sample code z from encoder output
4. Run sampled code through **decoder** to get a distribution over data samples
5. **Original input data should be likely under the distribution output from step 4**
6. Can sample a reconstruction from step 4

Problem: Can't differentiate through sampling process!



Reparameterization trick

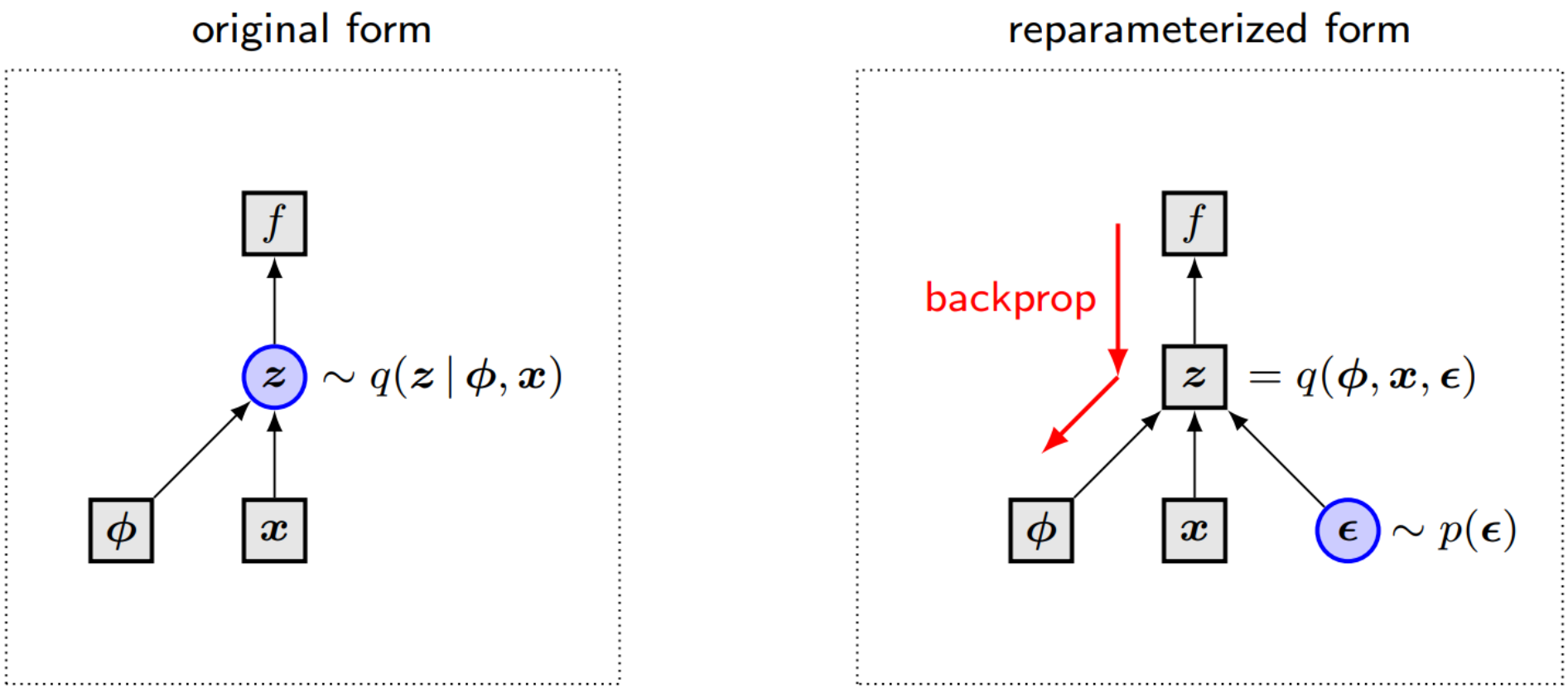
- idea: move sampling to an input
- given $\boldsymbol{\mu}(\boldsymbol{x})$ and $\boldsymbol{\Sigma}(\boldsymbol{x})$ (i.e. mean and covariance of $q(\boldsymbol{z}|\boldsymbol{x})$):
  - instead of sampling $\boldsymbol{z}^{(i,l)} \sim \mathcal{N}(\boldsymbol{\mu}(\boldsymbol{x}^{(i)}), \boldsymbol{\Sigma}(\boldsymbol{x}^{(i)}))$ directly
  - we can compute $\boldsymbol{z}^{(i,l)}$ by
  1. first sampling $\boldsymbol{\epsilon}^{(l)} \sim \mathcal{N}(\boldsymbol{z}; \mathbf{0}, \mathbf{I})$
  2. then computing

$$\boldsymbol{z}^{(i,l)} = \boldsymbol{\mu}(\boldsymbol{x}^{(i)}) + \boldsymbol{\sigma}(\boldsymbol{x}^{(i)}) \odot \boldsymbol{\epsilon}^{(l)}$$

    - $\odot$: element-wise multiplication

  - $\boldsymbol{z}^{(i,l)}$ has the same distribution as before, but now we can do backprop

Reparameterization trick



Variational Autoencoders: Data Generation

After training, we can generate new data!

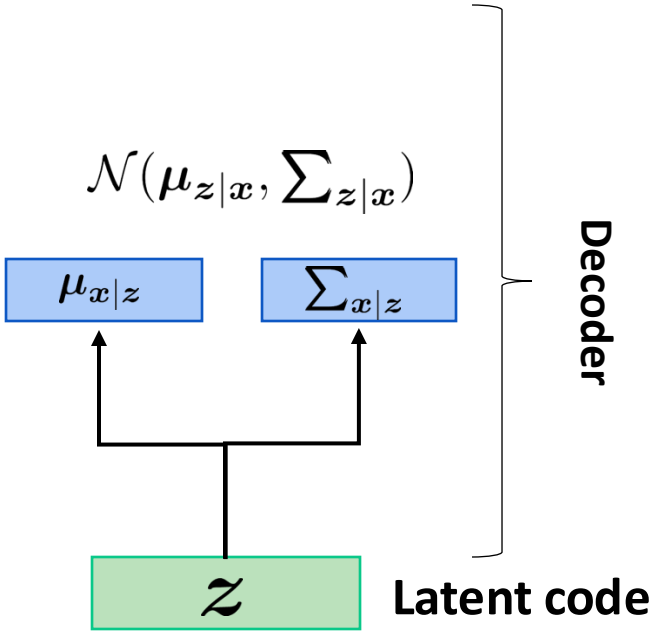
1. Sample code $\boldsymbol{z}$ from prior $p(\boldsymbol{z})$

$\boldsymbol{z}$ **Latent code**

Variational Autoencoders: Data Generation

After training, we can generate new data!

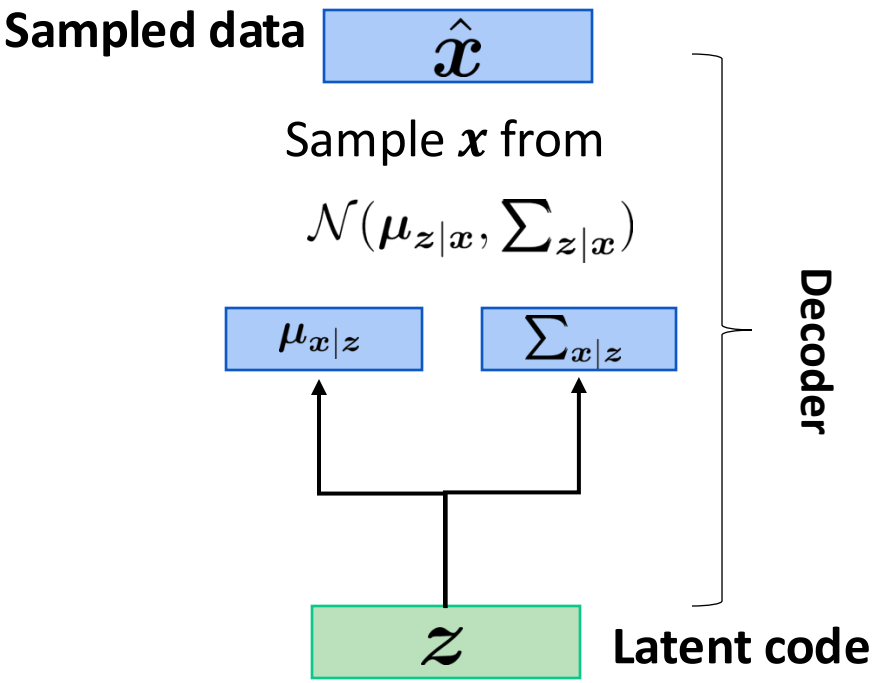
1. Sample code $\boldsymbol{z}$ from prior $p(\boldsymbol{z})$
2. Run sampled code $\boldsymbol{z}$ through **decoder** to get a distribution over data $\boldsymbol{x}$



Variational Autoencoders: Data Generation

After training, we can generate new data!

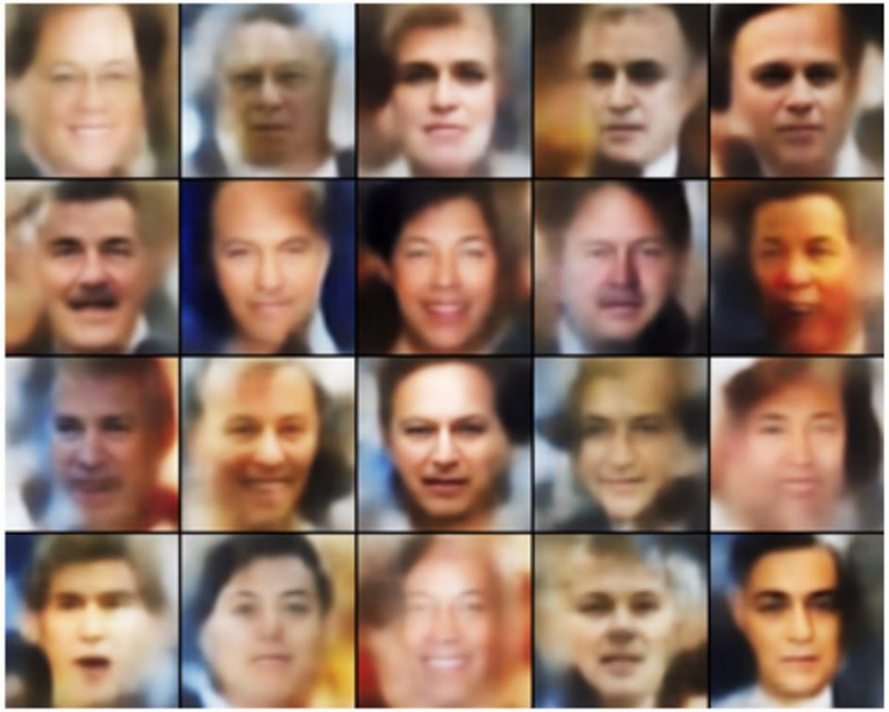
1. Sample code $\boldsymbol{z}$ from prior $p(\boldsymbol{z})$
2. Run sampled code $\boldsymbol{z}$ through **decoder** to get a distribution over data $\boldsymbol{x}$
3. Sample from distribution found in step 2 to generate data



Variational Autoencoders: Data Generation

32x32 CIFAR-10

Labeled Faces in the Wild



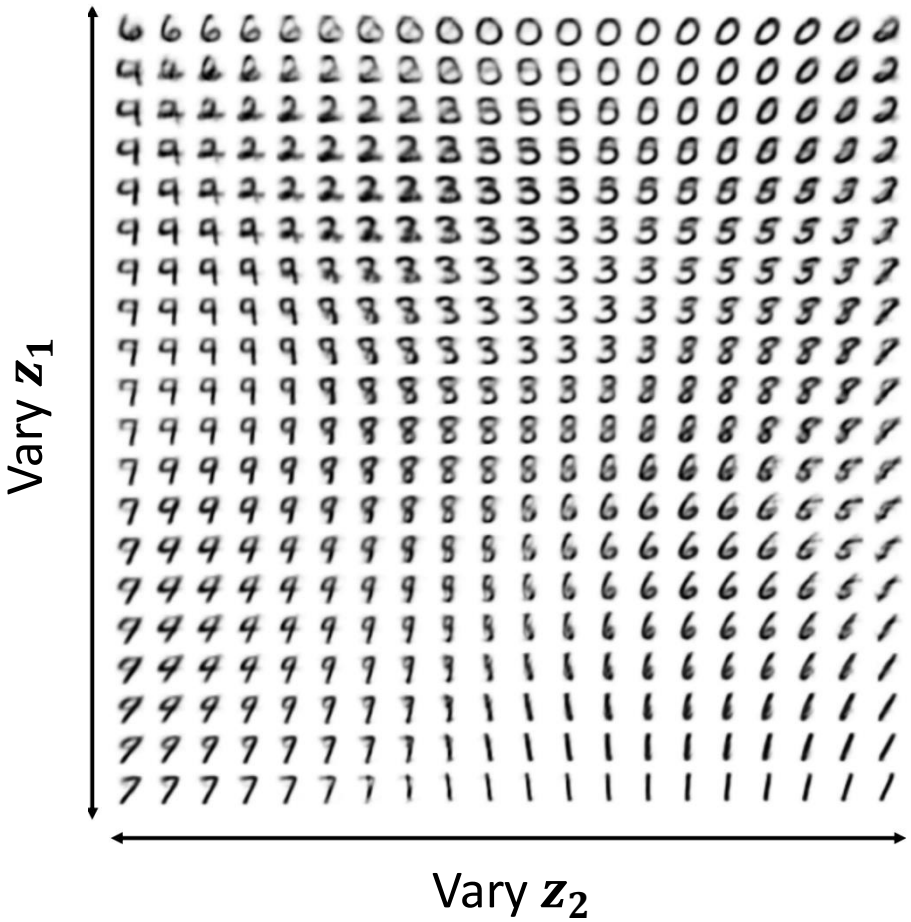
Figures from (L) Dirk Kingma et al. 2016; (R) Anders Larsen et al. 2017.

Generated Samples

Variational Autoencoders: Data Generation

The diagonal prior on $p(\boldsymbol{z})$ causes dimensions of $\boldsymbol{z}$ to be independent.

"Disentangling factors of variation"

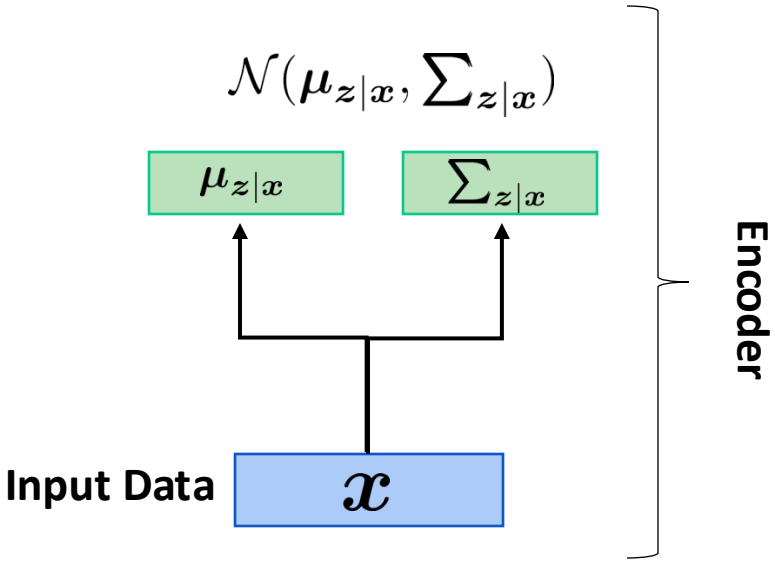


Kingma and Welling, Auto-Encoding Variational Bayes, ICLR 2014

Variational Autoencoders

After training, we can **edit images**

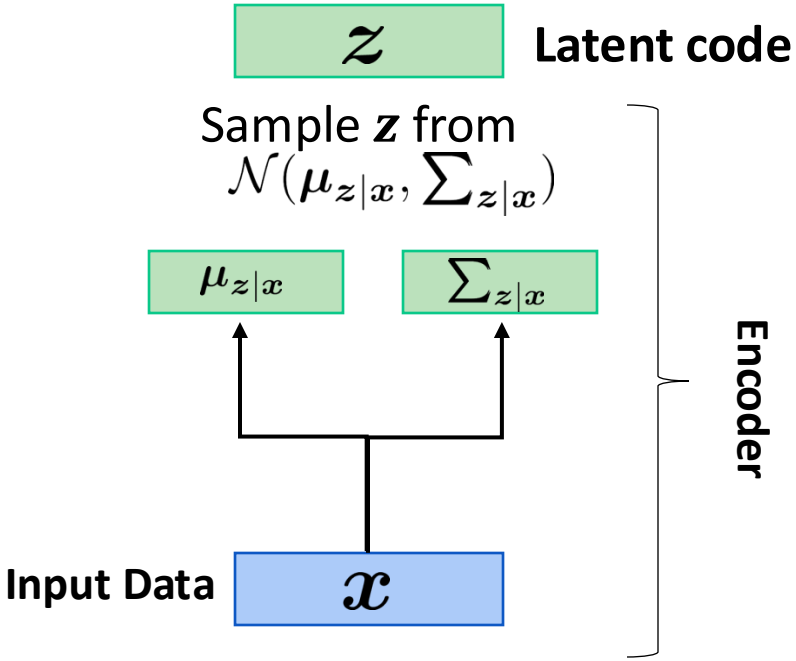
1. Run data through **encoder** to get a distribution over latent codes



Variational Autoencoders

After training, we can **edit images**

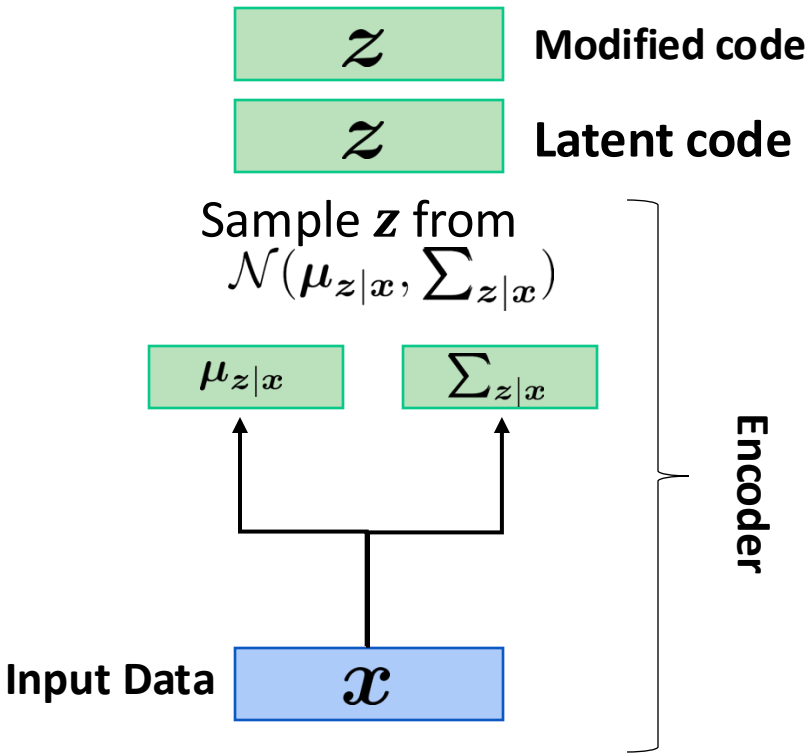
1. Run data through **encoder** to get a distribution over latent codes
2. Sample code $\boldsymbol{z}$ from encoder output



Variational Autoencoders

After training, we can **edit images**

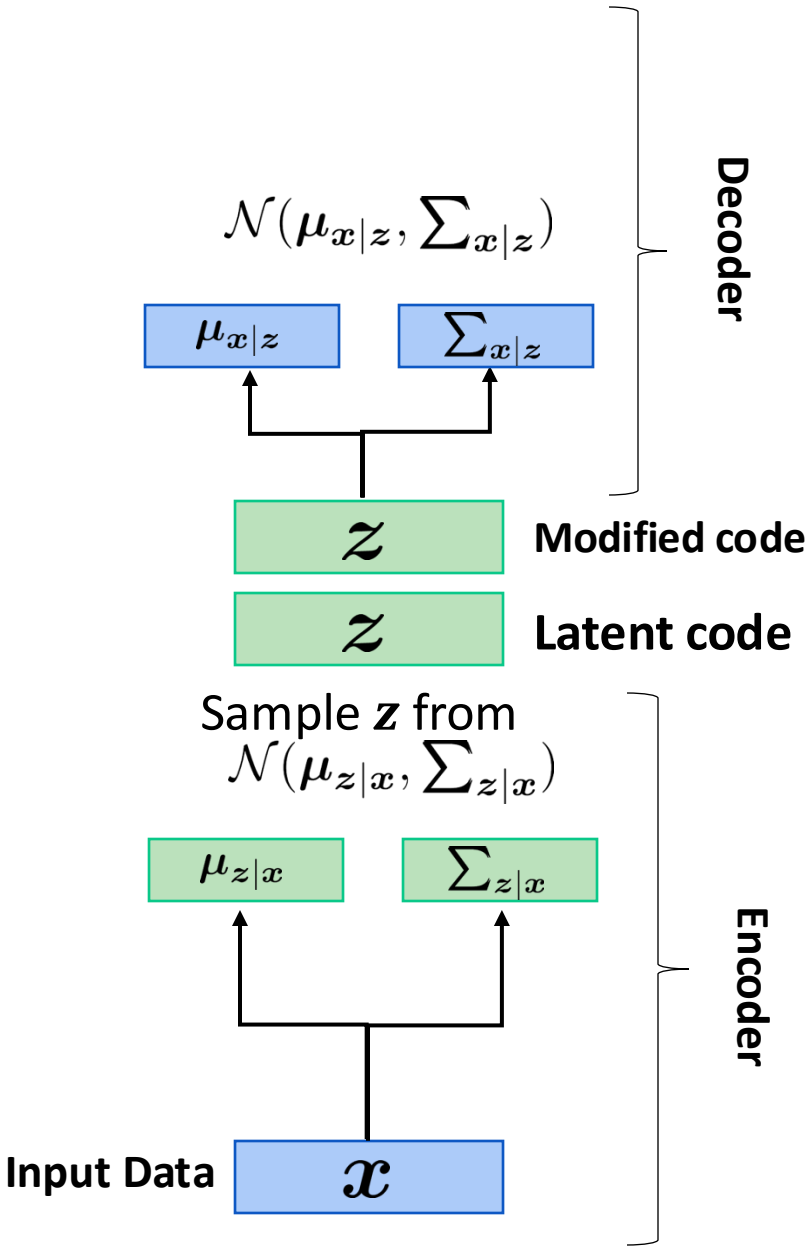
1. Run data through **encoder** to get a distribution over latent codes
2. Sample code $\boldsymbol{z}$ from encoder output
3. Modify some dimensions of sampled code



Variational Autoencoders

After training, we can **edit images**

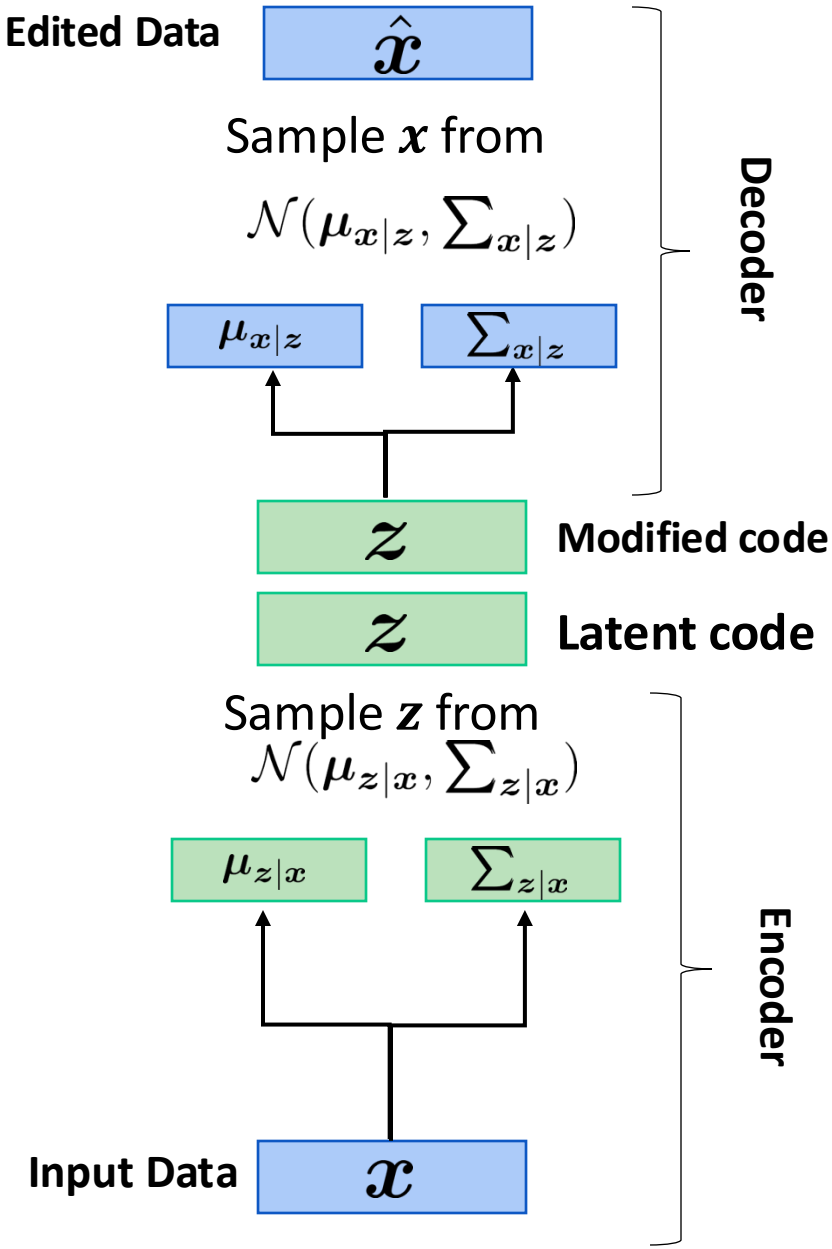
1. Run data through **encoder** to get a distribution over latent codes
2. Sample code $\boldsymbol{z}$ from encoder output
3. Modify some dimensions of sampled code
4. Run modified $\boldsymbol{z}$ through decoder to get a distribution over data sample



Variational Autoencoders

After training, we can **edit images**

1. Run data through **encoder** to get a distribution over latent codes
2. Sample code $\boldsymbol{z}$ from encoder output
3. Modify some dimensions of sampled code
4. Run modified $\boldsymbol{z}$ through decoder to get a distribution over data sample
5. Sample new data from step 4



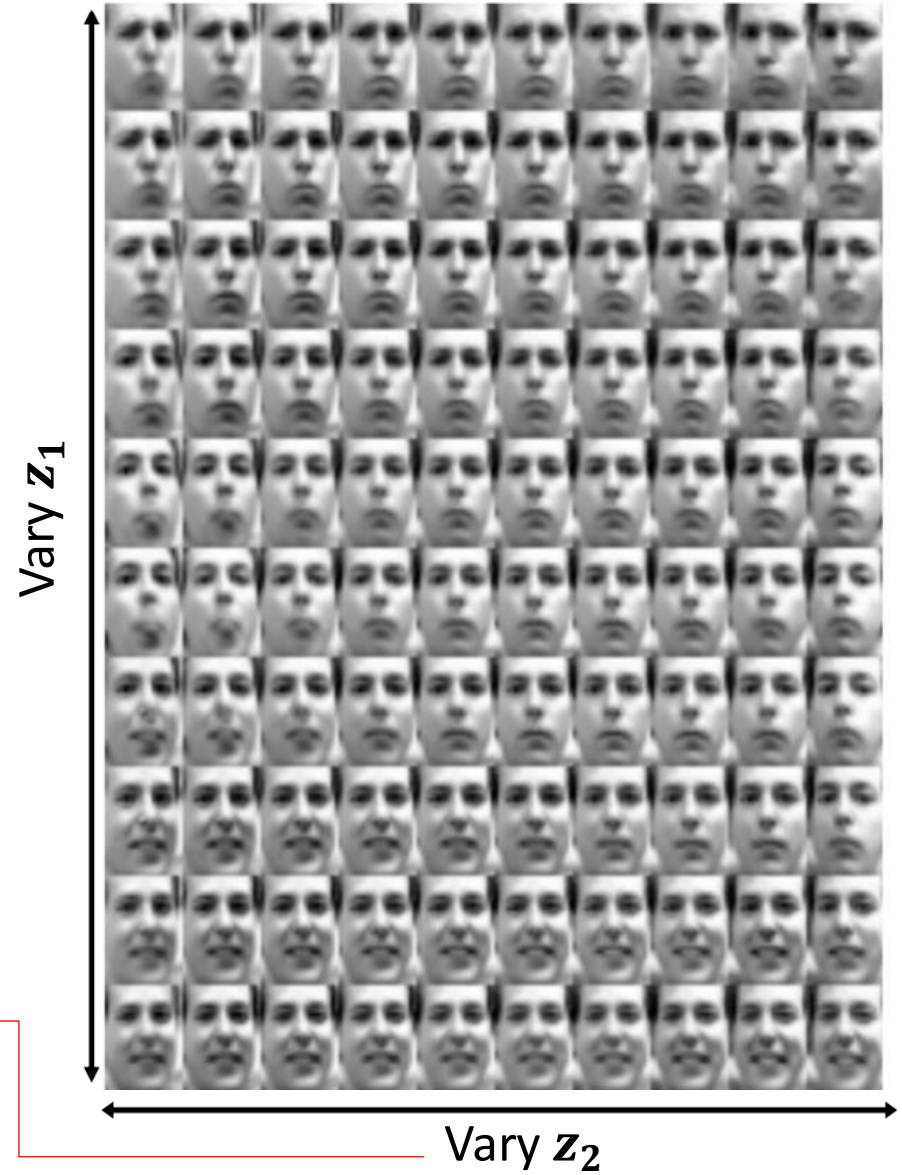
Variational Autoencoders: Image Editing

The diagonal prior on $p(\boldsymbol{z})$ causes dimensions of $\boldsymbol{z}$ to be independent.

"Disentangling factors of variation"

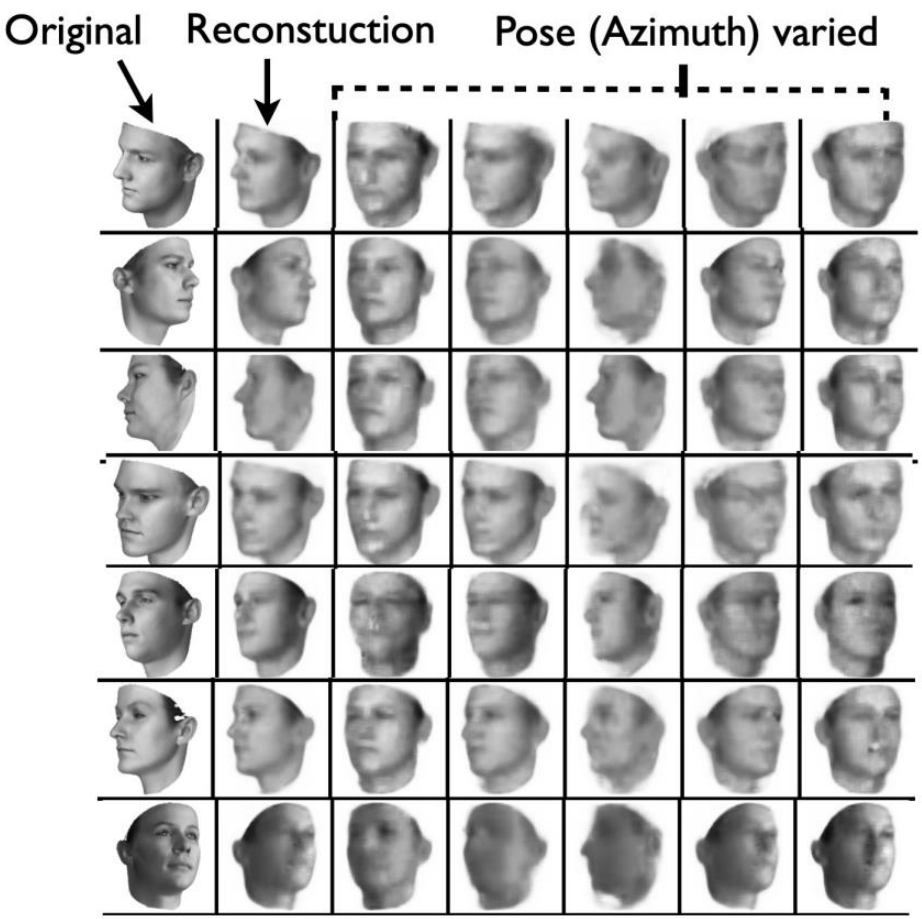
Degree of smile

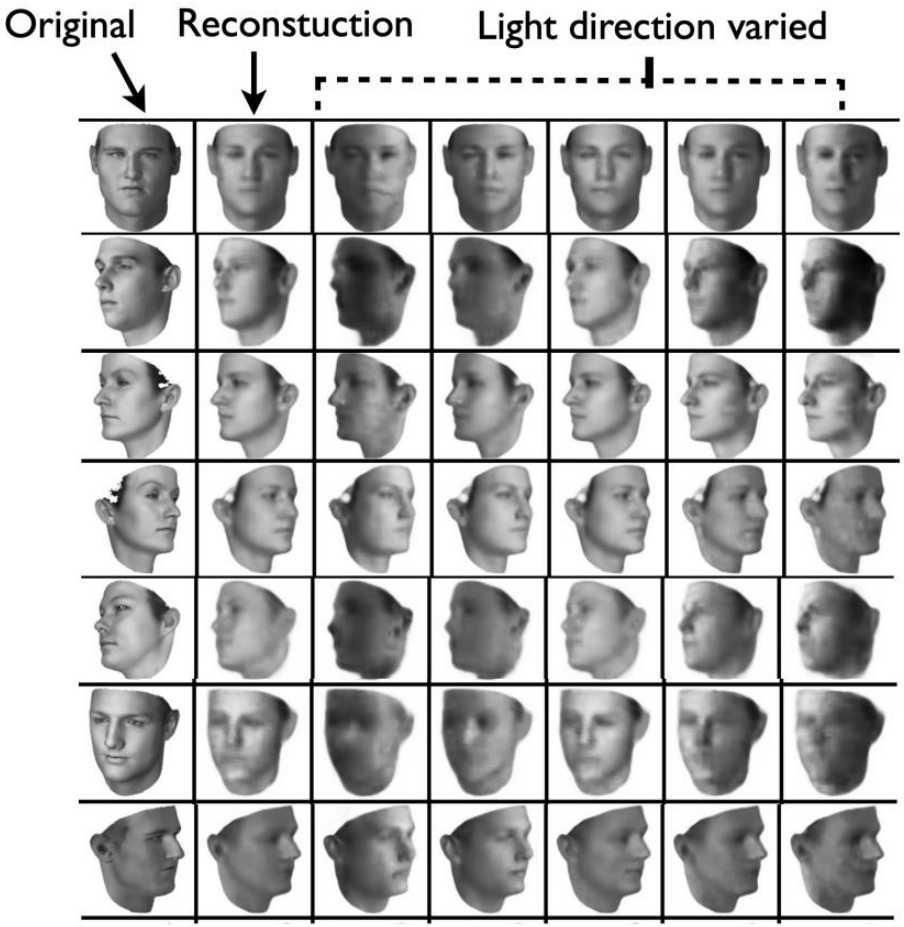
Head pose



Kingma and Welling, Auto-Encoding Variational Bayes, ICLR 2014

Variational Autoencoders: Image Editing

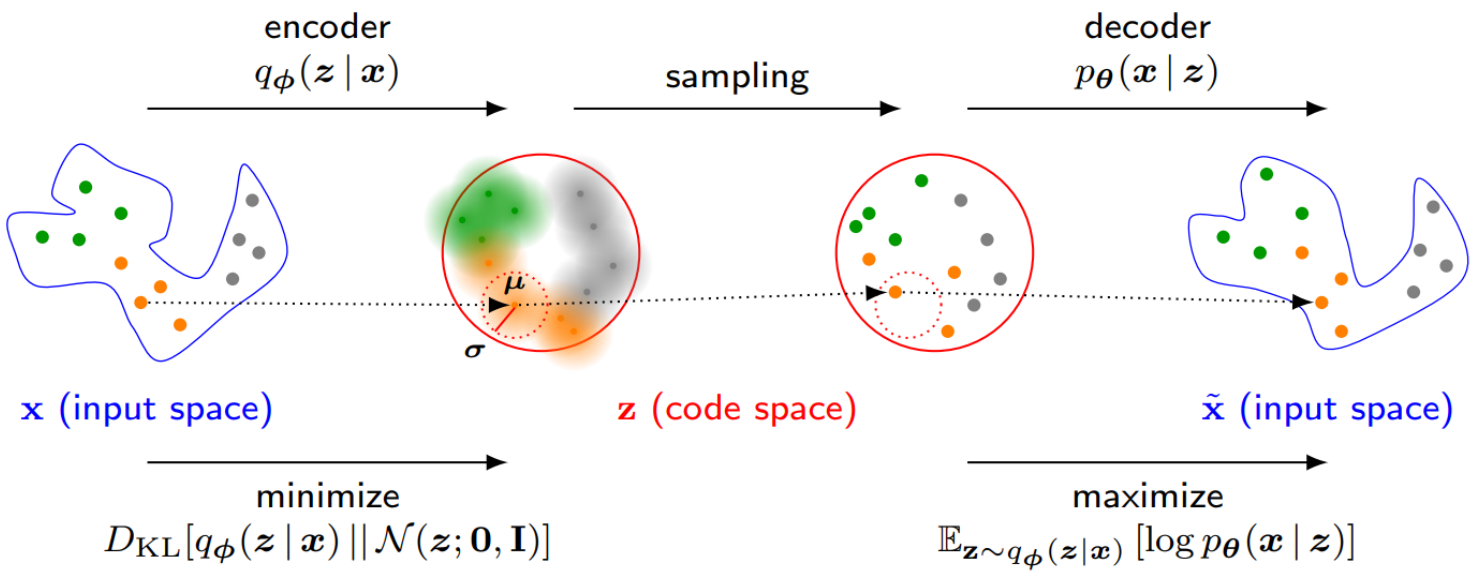




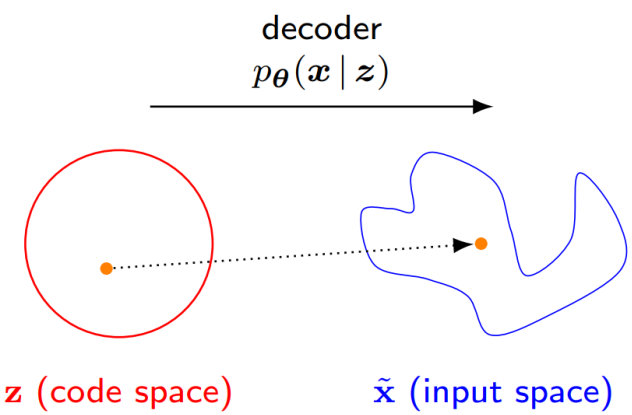
Kulkarni et al, "Deep Convolutional Inverse Graphics Networks", NeurIPS 2014

Variational Autoencoder: Summary

- training



- Sample generation
  - $\boldsymbol{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$: easy to sample from



Variational Autoencoder: Summary

- Probabilistic spin to traditional autoencoders => allows generating data
- Defines an intractable density => derive and optimize a (variational) lower bound
- Pros:
  - Principled approach to generative models
  - Allows inference of $\boldsymbol{q}(\boldsymbol{z}|\boldsymbol{x})$, can be useful feature representation for other tasks
- Cons:
  - Maximizes lower bound of likelihood: okay, but not as good evaluation as PixelRNN/PixelCNN
  - Samples blurrier and lower quality compared to state-of-the-art (GANs)
- Active areas of research:
  - More flexible approximations, e.g. richer approximate posterior instead of diagonal Gaussian, e.g., Gaussian Mixture Models (GMMs)
  - Incorporating structure in latent variables, e.g., Categorical Distributions